支撑材料文件清单与完整源程序

本文件用于并入论文附录。正文仍以主论文为准，本文件不替代主论文。程序按支撑材料中的实际文件完整列出，运行方式见使用说明。

附录：支撑材料文件清单

文件（相对于支撑材料根目录）
AI 工具使用详情.pdf
code/energy_paths.tex
code/explore_data.py
code/export_results.py
code/fig_data.py
code/fig_q1.py
code/fig_q2_sensitivity.py
code/fig_q3_days.py
code/fig_q3_quantile.py
code/framework.tex
code/prepare_inputs.py
code/prepare_source_tables.py
code/q1.py
code/q2.py
code/q3.py
code/q3_base.py
code/q4.py
code/q4_tree.py
code/q4_update.py
code/reproduce.py
code/source_map.json
code/validation.py
data/actual_power.csv.gz
data/actual_price.csv.gz
data/day_reference.csv.gz
data/inputs.npz
data/pv_forecasts.csv.gz
data/q1_config.json
data/raw_hashes.json
evidence/export_check.json
evidence/extra_reproduction_check.json
evidence/Q1_storage_sensitivity.csv
evidence/Q1_strategy_comparison.csv
evidence/Q2_annual_candidates.csv
evidence/Q2_refinement.csv

文件（相对于支撑材料根目录）
evidence/Q2_validation.csv
evidence/Q3_final_comparison.csv
evidence/Q4_final_comparison.csv
evidence/Q4_validation.csv
evidence/reproduction_check.json
evidence/workbook_readback.json
evidence/参考文献来源.tex
figure_inputs/derived_values.json
figure_inputs/eda/forecast_error_by_lead.csv
figure_inputs/eda/monthly_summary.csv
figure_inputs/eda/weekday_summary.csv
figure_inputs/q1/outputs/schedules/C03-p07.csv
figure_inputs/q1/outputs/schedules/C04.csv
figure_inputs/q1/outputs/schedules/C10-p05.csv
figure_inputs/q1/outputs/summary.json
figure_inputs/q2/refinement/comparison.csv
figure_inputs/q2/supplement/comparison.csv
figure_inputs/q2/validation/comparison.csv
figure_inputs/q3_daily.csv
figures/data_compact.pdf
figures/forecast_compact.pdf
figures/model.pdf
figures/q1_1.png
figures/q1_daily_dispatch.pdf
figures/q3_final_days.pdf
figures/q3_final_quantile.pdf
figures/sensitivity.png
figures/weiwang.png
figures/图源说明.md
requirements.txt
result1.xlsx
result2.xlsx
result3.xlsx
result4-2.xlsx
result4-3.xlsx
results/daily_costs.csv
results/Q1.csv
results/Q2.npz
results/Q3.npz
results/Q3_adjustments.csv.gz
results/Q4-2.npz
results/Q4-3.npz
results/Q4-3_adjustments.csv.gz

文件（相对于支撑材料根目录）
templates/result1.xlsx
templates/result2.xlsx
templates/result3.xlsx
templates/result4-2.xlsx
templates/result4-3.xlsx
使用说明.md
文件清单.md
SHA256SUMS.txt

附录：完整源程序

energy_paths.tex

```
\documentclass{standalone}
\usepackage[UTF8]{ctex}
\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning}
\begin{document}
% Q1 energy-path definition: report 12.2, 15; external-side kWh.
\begin{tikzpicture}[>=Stealth, node distance=16mm, every node/.style={font=\small},
  box/.style={draw=black,rounded corners=2pt,minimum width=23mm,minimum height=10mm,align=center},
  lab/.style={fill=white,inner sep=2pt}]
\node[box] (grid) at (0,1.25) {外部电网};
\node[box] (pv) at (0,-1.25) {光伏发电};
\node[box] (battery) at (5,0) {储能设备\\$S_{k-1}\longrightarrow S_k$};
\node[box] (load) at (10,1.25) {小区负载};
\node[box] (waste) at (10,-1.25) {未利用光伏};
\draw[->] (grid.east) -- node[lab,above] {$n_k-z_k$} (load.west);
\draw[->] (grid.south east) -- node[lab,above,sloped] {$x_k$} (battery.north west);
\draw[->] (pv.north east) -- node[lab,above,sloped] {$y_k$} (battery.south west);
\draw[->] (battery.north east) -- node[lab,below,sloped] {$z_k$} (load.south west);
\draw[->] (pv.south) -- (0,-2.65) -- node[lab,below] {$m_k$：光伏直供} (11.9,-2.65) -- (11.9,1.25) --
↪  (load.east);
\draw[->] (pv.east) -- node[lab,pos=.70,above] {$w_k=r_k-y_k$} (waste.west);
\end{tikzpicture}

\end{document}
```

explore_data.py

```
"""Descriptive EDA of supplied labels; no forecasting, dispatch or energy integration."""
from pathlib import Path
import argparse
import hashlib
import json
import platform
import numpy as np
import pandas as pd

PROJECT = Path(__file__).resolve().parents[1]
```

```
SOURCE = PROJECT / 'data'
SIGNALS = ['load_kw', 'pv_actual_kw', 'net_load_kw', 'price_yuan_per_kwh']


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def stats(values):
    a = np.asarray(values, dtype=float)
    return dict(n=len(a), mean=float(a.mean()), std=float(a.std(ddof=1)),
                min=float(a.min()), p10=float(np.quantile(a,.1)),
                median=float(np.median(a)), p90=float(np.quantile(a,.9)),
                max=float(a.max()), zero_count=int((a==0).sum()))


def lagcorr(a, lag):
    a = np.asarray(a, dtype=float)
    return float(np.corrcoef(a[:-lag], a[lag:])[0,1])


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--output', default='reproduced/eda')
    args = ap.parse_args()
    out = (PROJECT / args.output).resolve()
    if not out.is_relative_to(PROJECT) or out.exists():
        raise ValueError('Use a new output directory inside the project.')
    raw_hashes = json.loads((SOURCE/'raw_hashes.json').read_text(encoding='utf-8'))
    input_hashes = {f.name: sha(f) for f in SOURCE.iterdir() if f.is_file()}
    ref = pd.read_csv(SOURCE/'day_reference.csv.gz')
    actual = pd.read_csv(SOURCE/'actual_power.csv.gz', parse_dates=['source_date'])
    price = pd.read_csv(SOURCE/'actual_price.csv.gz', parse_dates=['source_date'])
    forecast = pd.read_csv(SOURCE/'pv_forecasts.csv.gz',
    ↪  parse_dates=['release_datetime','target_datetime'])
    keys = ['source_date','source_minute']
    data = actual.merge(price[keys+['price_yuan_per_kwh']],on=keys,validate='one_to_one')
    data = data.sort_values(keys).reset_index(drop=True)
    assert len(data)==52560 and len(ref)==144 and len(forecast)==35040
    assert data.groupby('source_date').size().eq(144).all()
    data['timestamp'] = data.source_date + pd.to_timedelta(data.source_minute, unit='min')
    assert data.timestamp.diff().dropna().eq(pd.Timedelta(minutes=10)).all()
    data['month'] = data.source_date.dt.month
    data['weekday'] = data.source_date.dt.dayofweek
    data['net_load_kw'] = data.load_kw - data.pv_actual_kw
    data['pv_exceeds_load'] = data.net_load_kw < 0
    ref['net_load_kw'] = ref.load_kw-ref.pv_forecast_kw
    out.mkdir(parents=True)
    ref.to_csv(out/'reference_profile.csv',index=False)
    data[keys+['month','weekday']+SIGNALS].to_csv(out/'time_series.csv',index=False)
    daily = data.groupby('source_date')[SIGNALS].mean()
    daily['pv_exceeds_load_fraction'] = data.groupby('source_date').pv_exceeds_load.mean()
    daily['peak_net_load_kw'] = data.groupby('source_date').net_load_kw.max()
    daily['minimum_net_load_kw'] = data.groupby('source_date').net_load_kw.min()
    daily.to_csv(out/'daily_summary.csv')
    monthly = data.groupby('month')[SIGNALS].mean()
    monthly['days'] = data.groupby('month').source_date.nunique()
    monthly['pv_exceeds_load_fraction'] = data.groupby('month').pv_exceeds_load.mean()
```

```
    monthly.to_csv(out/'monthly_summary.csv')
    profile = data.groupby('source_minute')[SIGNALS].agg(['mean',lambda s:s.quantile(.1),lambda s:s.quantile(.9)])
    profile.columns = [f'{c}_{["mean","p10","p90"][i%3]}' for i,(c,_) in enumerate(profile.columns)]
    profile.to_csv(out/'intraday_summary.csv')
    mp = data.groupby(['month','source_minute'])[SIGNALS].mean()
    mp.to_csv(out/'month_intraday.csv')
    weekdays = data.groupby('weekday')[SIGNALS].mean()
    weekdays['days'] = data.groupby('weekday').source_date.nunique()
    weekdays.to_csv(out/'weekday_summary.csv')
    lag_rows = []
    summary_lags = {}
    for col in SIGNALS:
        # Subtract source-month x time-label mean only for retrospective diagnostics.
        residue = data[col]-data.groupby(['month','source_minute'])[col].transform('mean')
        summary_lags[col] = {str(h): {'raw':lagcorr(data[col],int(h*6)),
        'month_time_residual':lagcorr(residue,int(h*6))} for h in [1,12,24,48,144,168,192,336]}
        for h in range(1,337):
            lag_rows.append(dict(signal=col,lag_hours=h,raw_correlation=lagcorr(data[col],h*6),residual_correlation=lagcorr(residue,h*6)))
    pd.DataFrame(lag_rows).to_csv(out/'lag_correlations.csv',index=False)
    matched = forecast.merge(data[['timestamp','pv_actual_kw']],left_on='target_datetime',right_on='timestamp',how='left',validate='many_to_one')
    assert matched.pv_actual_kw.isna().sum()==36
    matched['error_kw'] = matched.pv_forecast_kw-matched.pv_actual_kw
    matched['abs_error_kw'] = matched.error_kw.abs()
    matched['target_hour'] = matched.target_datetime.dt.hour
    matched['release_hour'] = matched.release_datetime.dt.hour
    valid = matched.dropna(subset=['pv_actual_kw']).copy()
    metrics=[]
    for name, frame in [('all',valid),('actual_positive',valid[valid.pv_actual_kw>0])]:
        for h,g in frame.groupby('lead_hours'):
            metrics.append(dict(scope=name,lead_hours=int(h),n=len(g),mae_kw=float(g.abs_error_kw.mean()),bias_kw=float(g.error_kw.mean()),rmse_kw=float(np.sqrt((g.error_kw**2).mean()))))
    pd.DataFrame(metrics).to_csv(out/'forecast_error_by_lead.csv',index=False)
    matched[['release_datetime','target_datetime','lead_hours','pv_forecast_kw','pv_actual_kw','error_kw','abs_error_kw','target_hour','release_hour']].to_csv(out/'forecast_matched.csv',index=False)
    # For each original midnight forecast targeting this day's h=7..24,
    # compare the original and the latest later published forecast strictly before target.
    initial = valid[(valid.release_hour==0)&(valid.lead_hours>=7)].copy()
    pairs=[]
    for r in initial.itertuples():
        pool=valid[(valid.target_datetime==r.target_datetime)&(valid.release_datetime>r.release_datetime)&(valid.release_datetime<r.target_datetime)]
        if pool.empty:
            continue
        new=pool.sort_values('release_datetime').iloc[-1]
        pairs.append(dict(target_datetime=r.target_datetime,actual_kw=r.pv_actual_kw,
                          initial_lead=r.lead_hours,latest_lead=int(new.lead_hours),
                          initial_forecast_kw=r.pv_forecast_kw,latest_forecast_kw=float(new.pv_forecast_kw),
                          initial_abs_error_kw=r.abs_error_kw,latest_abs_error_kw=float(new.abs_error_kw)))
    pairs=pd.DataFrame(pairs)
    pairs.to_csv(out/'forecast_same_target_pairs.csv',index=False)
```

```
    paired_stats={}
    for name,g in [('all',pairs),('actual_positive',pairs[pairs.actual_kw>0])]:
        diff=g.latest_abs_error_kw-g.initial_abs_error_kw
        paired_stats[name]=dict(n=len(g),initial_mae_kw=float(g.initial_abs_error_kw.mean()),latest_mae_kw=float(g.latest_abs_error_kw.mean()),improved_fraction=float((diff<0).mean()),worse_fraction=float((diff>0).mean()),tie_fraction=float((diff==0).mean()))
    def extreme_row(frame,col,which):
        r=frame.loc[getattr(frame[col],'idx'+which)()]
        return {'value':float(r[col]),'source_date':str(r.get('source_date','representative')),'source_minute':int(r.source_minute)}
    shapes={}
    for col in SIGNALS:
        matrix=data.pivot(index='source_date',columns='source_minute',values=col).to_numpy()
        corrs=np.array([np.corrcoef(matrix[i],matrix[i+1])[0,1] for i in range(364)])
        shapes[col]=stats(corrs)
    refstats={col:stats(ref[col]) for col in ['load_kw','pv_forecast_kw','net_load_kw','price_yuan_per_kwh']}
    refstats['positive_pv_labels'] = ref.loc[ref.pv_forecast_kw>0,'source_minute'].tolist()
    refstats['surplus_labels'] = ref.loc[ref.net_load_kw<0,'source_minute'].tolist()
    refstats['extrema']={col:{w:extreme_row(ref,col,w) for w in ['min','max']} for col in ['load_kw','pv_forecast_kw','net_load_kw','price_yuan_per_kwh']}
    price_spread=data.groupby('source_date').price_yuan_per_kwh.agg(['min','max'])
    price_spread['ratio']=price_spread['max']/price_spread['min']
    summary=dict(version='EDA-v1',scope='retrospective_descriptive_only',python=platform.python_version(),numpy=np.__version__,pandas=pd.__version__,script_sha256=sha(Path(__file__)),input_sha256=input_hashes,raw_sha256=raw_hashes,
                 reference=refstats,annual={c:stats(data[c]) for c in SIGNALS},
                 january_only={c:stats(data.loc[data.month==1,c]) for c in SIGNALS},
                 monthly=monthly.reset_index().to_dict('records'),lag_correlations=summary_lags,adjacent_day_shape_correlation=shapes,
                 annual_extrema={c:{w:extreme_row(data,c,w) for w in ['min','max']} for c in SIGNALS},
                 surplus=dict(sample_fraction=float(data.pv_exceeds_load.mean()),days_with_surplus=int((daily.pv_exceeds_load_fraction>0).sum()),max_surplus_kw=float(-data.net_load_kw.min())),
                 price_daily_max_min_ratio=stats(price_spread.ratio),forecast=dict(total=len(forecast),matched=len(valid),unmatched=36,actual_positive_count=int((valid.pv_actual_kw>0).sum()),mae_all_kw=float(valid.abs_error_kw.mean()),mae_actual_positive_kw=float(valid.loc[valid.pv_actual_kw>0,'abs_error_kw'].mean()),same_target=paired_stats),
                 caveats=['Source-label arithmetic means are not kWh or final interval semantics.','One year supports seasonal differences, not verified interannual periodicity.','Full-year diagnostics are hindsight; no future information is authorized for decisions.','Forecast metrics are at exact hourly target labels with no interpolation; economic value is not measured.','Month x time residuals are retrospective centering, not a trained forecasting model.'])
    assert input_hashes=={f.name:sha(f) for f in SOURCE.iterdir() if f.is_file()}
    # Raw file identifiers are retained; supplied CSV inputs are rechecked above.
    summary['source_unchanged']=True
    (out/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')
    print(json.dumps({'output':str(out),'monthly':summary['monthly'],'surplus':summary['surplus'],'forecast':summary['forecast'],'lags':summary_lags},ensure_ascii=False,indent=2))


if __name__=='__main__':
    main()
```

export_results.py

```
"""Export saved final results into the five official workbooks and check balances."""
from pathlib import Path
from datetime import datetime,timedelta
from copy import copy,deepcopy
import csv,gzip,json
import numpy as np
import pandas as pd
import openpyxl
from openpyxl.packaging.core import DocumentProperties
ROOT=Path(__file__).resolve().parents[1]
EXPECTED={'Q1':35126.85,'Q2':13583810.84,'Q3':13014172.33,'Q4-2':14248570.87,'Q4-3':13695929.70}
EXEC_NAMES=['storage_start_kwh','storage_end_kwh','received_kwh','grid_load_kwh','pv_load_kwh','grid_charge_kwh','pv_charge_kwh','discharge_kwh','emergency_kwh','unused_order_kwh','unused_pv_kwh','load_kwh']
def time_label(k):return f'{k//6:02d}:{k%6*10:02d}'
def metadata(w,template_name):
 w.properties=DocumentProperties(creator='',lastModifiedBy='',title='',description='',created=datetime(2026,9,13),modified=datetime(2026,9,13))
 for s in w:
  for row in s:
   for c in row:
    c.comment=None
    if isinstance(c.value,(int,float)):c.number_format='0.00'
 template=openpyxl.load_workbook(ROOT/'templates'/template_name)
 format_workbook(w,template)
 template.close()

def format_workbook(w,template):
 """Restore template styles, extending daily blocks without changing any values."""
 cache={}
 for s in w:
  t=template[s.title]
  for attr in ['sheet_format','sheet_properties','views','auto_filter',
               'page_setup','page_margins','print_options']:
   setattr(s,attr,deepcopy(getattr(t,attr)))
  s.print_title_rows=t.print_title_rows
  s.print_title_cols=t.print_title_cols
  s.print_area=str(t.print_area) if t.print_area else ''
  s.row_dimensions.clear();s.column_dimensions.clear()
  for key,dim in t.column_dimensions.items():
   s.column_dimensions[key]=copy(dim);s.column_dimensions[key].parent=s
  for key,dim in t.row_dimensions.items():
   s.row_dimensions[key]=copy(dim);s.row_dimensions[key].parent=s
  annual_storage=s.title==' 充放电量' and s.max_column==6
  for row in s:
   r=row[0].row;single=False
   if r==1:tr=1
   elif annual_storage:tr=2+(r-2)%6
   elif s.title==' 紧急购电量':
    date=s.cell(r,1).value
    first=r==2 or s.cell(r-1,1).value!=date
    last=r==s.max_row or s.cell(r+1,1).value!=date
    tr=2 if first else (4 if last else 3)
    single=first and last
   else:tr=r if r<=t.max_row else 2
   if tr in t.row_dimensions:
```

```
    s.row_dimensions[r]=copy(t.row_dimensions[tr])
    s.row_dimensions[r].index=r;s.row_dimensions[r].parent=s
   for c in row:
    ref=t.cell(tr,c.column)
    # All retained dates use the template's date format, including continuation rows.
    number_format=t.cell(2,1).number_format if isinstance(c.value,datetime) else ref.number_format
    key=(s.title,ref.style_id,number_format,single,c.column if single else 0)
    if key not in cache:
     for attr in ['font','fill','border','alignment','protection']:
      setattr(c,attr,copy(getattr(ref,attr)))
     c.number_format=number_format
     if single:
      border=copy(c.border);border.bottom=copy(t.cell(4,c.column).border.bottom)
      c.border=border
     cache[key]=copy(c._style)
    c._style=copy(cache[key])
def numeric(s,r,c,v):
 if abs(v)<1e-10:v=0.
 s.cell(r,c,float(v)).number_format='0.00'
def clear_data(s):
 for row in s.iter_rows(min_row=2):
  for c in row:c.value=None
def fill_orders(s,orders,fee):
 for i,order in enumerate(orders):
  r=i+2
  for k,v in enumerate(order):numeric(s,r,145 if k==0 else k+1,v)
  numeric(s,r,146,order.sum());numeric(s,r,147,fee[i])
 s.freeze_panes='B2'
 # The official template's last header is retained; its same-date slot-0 meaning is documented.
def fill_storage(s,ex):
 styles=[copy(c._style) for c in s[2]];clear_data(s)
 for i,day in enumerate(ex):
  date=datetime(2025,2,1)+timedelta(days=i)
  for g in range(6):
   r=2+i*6+g
   for j,style in enumerate(styles,1):s.cell(r,j)._style=copy(style)
   s.cell(r,1,date).number_format='yyyy-mm-dd'
   s.cell(r,2,f'{4*g}:00-{4*(g+1)}:00')
   numeric(s,r,3,day[g*24:(g+1)*24,5:7].sum());numeric(s,r,4,day[g*24:(g+1)*24,7].sum())
   if g<2:s.cell(r,5,'0:00' if g==0 else '24:00');numeric(s,r,6,day[0,0] if g==0 else day[-1,1])
 s.freeze_panes='C2'
def fill_emergency(s,ex):
 clear_data(s);r=2;events=0
 for i,day in enumerate(ex):
  k=0;found=False;date=datetime(2025,2,1)+timedelta(days=i)
  while k<144:
   if day[k,8]<=1e-8:k+=1;continue
   end=k+1
   while end<144 and day[end,8]>1e-8:end+=1
   s.cell(r,1,date).number_format='yyyy-mm-dd';s.cell(r,2,f'{time_label(k)}-{time_label(end)}');nume
   ↪ ric(s,r,3,day[k:end,8].sum())
   k=end;r+=1;events+=1;found=True
  if not found:s.cell(r,1,date).number_format='yyyy-mm-dd';s.cell(r,2,' 无');numeric(s,r,3,0);r+=1
 s.column_dimensions['A'].width=15;s.column_dimensions['B'].width=22;s.column_dimensions['C'].width=
 ↪ 18
 return events
def main():
 src=np.load(ROOT/'data/inputs.npz');report={};dailyrows=[]
```

```
q1=pd.read_csv(ROOT/'results/Q1.csv',float_precision='round_trip')
w=openpyxl.load_workbook(ROOT/'templates/result1.xlsx')
for k,v in enumerate(q1.purchase_kwh):numeric(w[' 计划购电量'],145 if k==0 else k+1,2,v)
for g in range(6):
 part=q1.iloc[g*24:(g+1)*24];numeric(w['充放电量'],g+2,2,part.charge_kwh.sum());numeric(w['充放电量'],g+2,3,part.discharge.sum())
numeric(w['充放电量'],2,5,q1.state_start.iloc[0]);numeric(w['充放电量'],3,5,q1.state_end.iloc[-1]);metadata(w,'result1.xlsx');w.save(ROOT/'result1.xlsx')
report['Q1']={'total_cost':float(q1.cost_yuan.sum()),'order_kwh':float(q1.purchase_kwh.sum())}
for label in ['Q2','Q3','Q4-2','Q4-3']:
 z=dict(np.load(ROOT/'results'/f'{label}.npz'));ex=z['execution'];rolling='versions' in z
 order=z['versions'][:,-1,:] if rolling else z['order'];zero=z['q0'] if rolling else order
 price=src['actual_prices'][31:365] if label.startswith('Q4') else np.tile(src['price'],(334,1))
 normal=(price*order).sum(1);adjust=(.5*price*abs(order-zero)).sum(1);emergency=(5*price*ex[:,:,8]).sum(1)
 cost=normal+adjust+emergency;assert abs(cost.sum()-EXPECTED[label])<.005,(label,cost.sum())
 balance=ex[:,:,3]+ex[:,:,4]+ex[:,:,7]+ex[:,:,8]-ex[:,:,11]
 storage=ex[:,:,1]-ex[:,:,0]-.9*(ex[:,:,5]+ex[:,:,6])+ex[:,:,7]/.9
 assert max(abs(balance).max(),abs(storage).max())<1e-6
 assert ex[:,:,:2].min()>=1200-1e-6 and ex[:,:,:2].max()<=10800+1e-6
 assert max((ex[:,:,5]+ex[:,:,6]).max(),ex[:,:,7].max())<=5000/6+1e-6
 assert np.max(abs(ex[1:,0,0]-ex[:-1,-1,1]))<1e-6
 if rolling:
  for a,r in enumerate([36,72,108],1):assert np.max(abs(z['versions'][:,a,:r]-z['versions'][:,a-1,:r]))<1e-7
 file={'Q2':'result2','Q3':'result3','Q4-2':'result4-2','Q4-3':'result4-3'}[label]
 w=openpyxl.load_workbook(ROOT/'templates'/f'{file}.xlsx');fill_orders(w['计划购电量'],zero,(price*zero).sum(1))
 if rolling:fill_orders(w[' 调整购电量'],order,normal+adjust)
 fill_storage(w['充放电量'],ex);events=fill_emergency(w['紧急购电量'],ex);metadata(w,f'{file}.xlsx');w.save(ROOT/f'{file}.xlsx')
 if rolling:
  with gzip.open(ROOT/'results'/f'{label}_adjustments.csv.gz','wt',encoding='utf-8-sig',newline='') as f:
   writer=csv.writer(f);writer.writerow(['date','release_time','interval_start','interval_end','midnight_kwh','revised_total_kwh','net_change_vs_midnight_kwh'])
   for i in range(334):
    date=str((datetime(2025,2,1)+timedelta(days=i)).date())
    for a,r in enumerate([0,36,72,108]):
     for k in range(r,144):writer.writerow([date,time_label(r),time_label(k),time_label(k+1),zero[i,k],z['versions'][i,a,k],z['versions'][i,a,k]-zero[i,k]])
 for i in range(334):dailyrows.append(dict(question=label,date=str((datetime(2025,2,1)+timedelta(days=i)).date()),midnight_order_kwh=float(zero[i].sum()),final_order_kwh=float(order[i].sum()),normal_cost=normal[i],adjustment_cost=adjust[i],emergency_cost=emergency[i],total_cost=cost[i],emergency_kwh=float(ex[i,:,8].sum()),initial_kwh=ex[i,0,0],final_kwh=ex[i,-1,1]))
 report[label]=dict(days=334,slots=48096,storage_groups=2004,emergency_events=events,total_cost=float(cost.sum()),normal_cost=float(normal.sum()),adjustment_cost=float(adjust.sum()),emergency_cost=float(emergency.sum()),energy_balance_max_error=float(abs(balance).max()),storage_max_error=float(abs(storage).max()))
pd.DataFrame(dailyrows).to_csv(ROOT/'results/daily_costs.csv',index=False,encoding='utf-8-sig',float_format='%.17g')
(ROOT/'evidence/export_check.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(report,ensure_ascii=False,indent=2))
if __name__=='__main__':main()
```

fig_data.py

```
"""Re-layout saved EDA only; no fitting or model changes."""
from pathlib import Path
import os,hashlib,json
P=Path(__file__).resolve().parents[1]
paper=P/'reproduced/plots'
(paper/'fig').mkdir(parents=True,exist_ok=True)
(paper/'scripts').mkdir(exist_ok=True)
os.environ.setdefault('MPLCONFIGDIR',str(P/'tmp/matplotlib'))
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd
source=P/'figure_inputs/eda'
monthly=pd.read_csv(source/'monthly_summary.csv')
weekday=pd.read_csv(source/'weekday_summary.csv')
errors=pd.read_csv(source/'forecast_error_by_lead.csv')
plt.rcParams.update({'font.family':'SimSun','font.size':11,'axes.unicode_minus':False,'pdf.fonttype':42,
 'axes.spines.top':False,'axes.spines.right':False,'legend.frameon':False,'savefig.facecolor':'white'})
out=paper/'fig'
fig,ax=plt.subplots(1,2,figsize=(7.2,2.6),layout='constrained')
ax[0].plot(monthly.month,monthly.load_kw,'o-',color='#b85518',label=' 负载',lw=1.4,ms=3)
ax[0].plot(monthly.month,monthly.pv_actual_kw,'s--',color='#2166ac',label=' 光伏',lw=1.4,ms=3)
ax[0].set(title='（a）负载与光伏的月平均水平',xlabel='月份',ylabel='平均功率 / kW',xticks=[1,3,5,7,9,12],ylim=(0,6500))
ax[0].legend(ncol=2,fontsize=10)
ax[1].bar(weekday.weekday,weekday.load_kw,color=['#b85518']*4+['#2166ac']*2+['#b85518'],width=.6)
ax[1].set(title='（b）不同星期位置的平均负载',xlabel='星期',ylabel='平均功率 / kW',xticks=list(range(7)),xticklabels=list('一二三四五六日'),ylim=(0,6500))
for a in ax:a.grid(axis='y',alpha=.18);a.set_axisbelow(True)
fig.savefig(out/'data_compact.pdf',bbox_inches='tight');fig.savefig(out/'data_compact.png',dpi=220,box_inches='tight');plt.close(fig)
fig,ax=plt.subplots(figsize=(6.4,2.25),layout='constrained')
for scope,label,color,style in [('all','全部匹配样本','#2166ac','--'),('actual_positive','实际光伏大于零','#b85518','-')]:
    g=errors[errors.scope==scope]
    ax.plot(g.lead_hours,g.mae_kw,label=label,color=color,ls=style,lw=1.5,marker='o',ms=2.5)
ax.set(xlabel=' 预报提前时间 / h',ylabel=' 平均绝对误差 / kW',xticks=[1,6,12,18,24],ylim=(0,None))
ax.legend(fontsize=10);ax.grid(axis='y',alpha=.18)
fig.savefig(out/'forecast_compact.pdf',bbox_inches='tight');fig.savefig(out/'forecast_compact.png',dpi=220,bbox_inches='tight');plt.close(fig)
files=[source/n for n in ['monthly_summary.csv','weekday_summary.csv','forecast_error_by_lead.csv']]
manifest={'sources':[{'path':str(p),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in files],
 'scope':'Saved descriptive statistics re-plotted without altering data or model; monthly and weekday means follow original source-date grouping.',
 'outputs':['fig/data_compact.pdf','fig/data_compact.png','fig/forecast_compact.pdf','fig/forecast_compact.png']}
(paper/'scripts/compact_figures_manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),'utf-8')
print('Saved compact figures')
```

fig_q1.py

```
from pathlib import Path
import json, csv, hashlib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
PROJECT=Path(__file__).resolve().parents[1]
HERE=PROJECT/'reproduced/plots/fig'
HERE.mkdir(parents=True,exist_ok=True)
plt.rcParams.update({'font.family':'SimSun','font.size':10,'axes.unicode_minus':False,
 'text.color':'black','axes.labelcolor':'black','xtick.color':'black','ytick.color':'black',
 'pdf.fonttype':42,'axes.spines.top':False,'axes.spines.right':False,
 'axes.axisbelow':True,'savefig.facecolor':'white','legend.frameon':False})
def save(fig, name):
    fig.canvas.draw()
    for ext in ('pdf','png'):
        kw = {'metadata':{'CreationDate':None,'ModDate':None}} if ext=='pdf' else {}
        fig.savefig(HERE/f'{name}.{ext}',dpi=300,bbox_inches='tight',**kw)
    fig.savefig(HERE/f'{name}_preview.png',dpi=120,bbox_inches='tight')
    plt.close(fig)

ROOT=PROJECT/'figure_inputs/q1'
def rows(path):
    with path.open(encoding='utf-8-sig',newline='') as f: return list(csv.DictReader(f))
summary=json.loads((ROOT/'outputs/summary.json').read_text('utf-8'))
selected={row['candidate']:row for row in summary['comparison']}
schedules={c:rows(ROOT/'outputs/schedules'/f"{row['key']}.csv") for c,row in selected.items()}
p=np.array([float(r['price']) for r in schedules['04']]); t=np.arange(144)/6
peak=int(p.argmax())
names_cn={'03':' 峰谷策略','04':' 线性规划','10':' 光伏预留策略'}
colors={'03':'#b65a32','04':'#1f66a5','10':'#32805a'}
styles={'03':'--','04':'-','10':':'}
fig=plt.figure(figsize=(8.2,9.1))
gs=fig.add_gridspec(4,1,height_ratios=[1,.21,1,1],left=.115,right=.975,bottom=.10,top=.94,hspace=.18)
axs=[fig.add_subplot(gs[r]) for r in (0,2,3)]
legend_ax=fig.add_subplot(gs[1]); legend_ax.axis('off')
axs[0].step(np.r_[t,24],np.r_[p,p[-1]],where='post',color='#444444',lw=1.5)
axs[0].set_ylabel(' 电价（元/kWh）'); axs[0].set_title(' 同一天，不同策略怎样安排电池和购电')
axs[0].axvline(t[peak],color='#999999',lw=.8,ls=':')
axs[0].annotate(' 当天最高价',xy=(t[peak],p[peak]),xytext=(16,p[peak]+.12),
    arrowprops={'arrowstyle':'->','color':'black'},fontsize=10)
axs[0].set_ylim(0,max(p)+.3)
for c in ['03','10','04']:
    rr=schedules[c]; ss=[float(rr[0]['state_start'])]+[float(r['state_end']) for r in rr]
    axs[1].plot(np.arange(145)/6,ss,label=names_cn[c],color=colors[c],ls=styles[c],lw=1.6)
    qq=np.array([float(r['purchase_kwh']) for r in rr])*6
    axs[2].step(np.r_[t,24],np.r_[qq,qq[-1]],where='post',label=names_cn[c],color=colors[c],ls=style
    ↪  s[c],lw=1.25)
for bound in [1200,10800]: axs[1].axhline(bound,color='#888888',ls=':',lw=.8)
axs[1].set_ylim(0,12000); axs[1].set_yticks([1200,6000,10800]); axs[1].set_ylabel(' 内部储电量（kWh）')
handles,labels=axs[1].get_legend_handles_labels()
legend_ax.legend(handles,labels,ncol=3,loc='center',fontsize=11); axs[2].set_ylabel('外网购电功率（kW）
↪  '); axs[2].set_xlabel('时刻（时）')
for ax in axs:
    ax.set_xlim(0,24); ax.set_xticks(range(0,25,2)); ax.grid(axis='y',alpha=.18)
```

```
fig.text(.06,.022,'同一代表日；两种规则各选27组参数中费用最低者。首尾均为6000 kWh，充放效率各90%。',fontsize=9)
for axis in axs[:2]: axis.tick_params(labelbottom=False)
save(fig,'03_Q1_daily_dispatch')
```

fig_q2_sensitivity.py

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
ROOT=Path(__file__).resolve().parents[1]
B=ROOT/'figure_inputs/q2'
out=ROOT/'reproduced/plots/fig';out.mkdir(parents=True,exist_ok=True)
read=lambda p:pd.read_csv(p)
def save(fig,name):
 for ext in ('png','pdf'):fig.savefig(out/f'{name}.{ext}',dpi=300,bbox_inches='tight')
 plt.close(fig)
plt.rcParams.update({'font.family':'SimSun', 'font.size':10, 'axes.unicode_minus':False,
    'text.color':'black', 'axes.labelcolor':'black', 'xtick.color':'black', 'ytick.color':'black',
    'pdf.fonttype':42, 'axes.spines.top':False, 'axes.spines.right':False,
    'axes.axisbelow':True, 'savefig.facecolor':'white'})
BLUE, GREEN, ORANGE, RED = '#386a94', '#5f8768', '#c39243', '#b34e4a'
allrows=pd.concat([read(B/f'{p}/comparison.csv') for p in ['refinement','validation','supplement']]).set_index('variant')
base=allrows.loc['error-decay70']
choices=[['selected-q60','error-decay70','selected-q80'],['error-decay70','weighted-error','error-decay90']]
fig,axes=plt.subplots(1,2,figsize=(7.3,3.1),sharey=True)
sens=[]
for j,(keys,labels,title) in enumerate(zip(choices,[['0.6','0.7','0.8'],['0.7','0.8','0.9']],['分位位置 α（固定 β=0.7）','时间衰减 β（固定 α=0.7）'])):
    rows=allrows.loc[keys];xs=np.arange(3)
    ordinary=(rows.ordinary_cost-base.ordinary_cost)/1e4
    emergency=(rows.emergency_cost-base.emergency_cost)/1e4
    total=(rows.total_cost-base.total_cost)/1e4
    axes[j].bar(xs-.16,ordinary,width=.29,color=BLUE,label=' 普通电费变化')
    axes[j].bar(xs+.16,emergency,width=.29,color=ORANGE,label=' 紧急电费变化')
    axes[j].plot(xs,total,'ko-',lw=1,ms=3,label=' 总电费变化')
    axes[j].axhline(0,color='black',lw=.7)
    axes[j].set(xticks=xs,xticklabels=labels,xlabel=title,ylim=(-40,55))
    axes[j].grid(axis='y',alpha=.15)
    for x,y in zip(xs,total):axes[j].annotate(f'{y:.2f}',(x,y),xytext=(0,6),textcoords='offset points',ha='center',fontsize=8)
    for key,l in zip(keys,labels):sens.append({'parameter':'alpha' if j==0 else 'beta','value':l,'variant':key,**allrows.loc[key,['ordinary_cost','emergency_cost','total_cost']].to_dict()})
axes[0].set_ylabel(' 相对 α=β=0.7 的费用变化 / 万元')
handles,labels=axes[0].get_legend_handles_labels();fig.legend(handles,labels,loc='upper center',ncol=3,frameon=False,fontsize=9)
fig.tight_layout(rect=(0,0,1,.88));save(fig,'sensitivity')
pd.DataFrame(sens).to_csv(out/'sensitivity_input.csv',index=False)
```

fig_q3_days.py

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
from matplotlib import pyplot as plt,font_manager
ROOT=Path(__file__).resolve().parents[1]
FIG=ROOT/'reproduced/plots/fig';FIG.mkdir(parents=True,exist_ok=True)
z=dict(np.load(ROOT/'results/Q3.npz'))
df=pd.read_csv(ROOT/'figure_inputs/q3_daily.csv')
truth=np.load(ROOT/'data/inputs.npz')['actual']
font_manager.findfont('SimSun', fallback_to_default=False)
plt.rcParams.update({'font.family': ['Times New Roman','SimSun'], 'font.size':11,
 'axes.titlesize':12, 'axes.labelsize':11, 'legend.fontsize':10,
 'axes.unicode_minus':False, 'pdf.fonttype':42, 'ps.fonttype':42,
 'text.color':'black','axes.labelcolor':'black','xtick.color':'black','ytick.color':'black',
 'axes.spines.top':False,'axes.spines.right':False,'axes.linewidth':.8,
 'savefig.facecolor':'white','figure.facecolor':'white','axes.axisbelow':True})
BLUE='#396e96'; GOLD='#bf913c'; GREEN='#62896c'; GRAY='#888888'
outputs=[]
def save(fig, name):
    for ext in ('pdf','png'):
        p=FIG/f'{name}.{ext}'
        kw={'metadata':{'CreationDate':None,'ModDate':None}} if ext=='pdf' else {'dpi':300}
        fig.savefig(p, bbox_inches='tight', pad_inches=.07, **kw); outputs.append(p)
    plt.close(fig)
def grid(ax): ax.grid(axis='y',color='#dddddd',lw=.5)

# Q3 monthly charges: sum raw daily bills, without selecting favorable months.
monthly=df.assign(month=pd.to_datetime(df.date).dt.month).groupby('month').sum(numeric_only=True)
fig,ax=plt.subplots(figsize=(9.1,3.3), layout='constrained')
x=monthly.index.to_numpy()
ax.bar(x,monthly.ordinary_cost/1e4,.63,color=BLUE,label=' 普通结算')
ax.bar(x,monthly.emergency_cost/1e4,.63,bottom=monthly.ordinary_cost/1e4,color=GOLD,label=' 紧急购电')
ax.set(xlabel=' 月份',ylabel=' 月度费用 / 万元',xticks=x,ylim=(0,180))
ax.legend(loc='upper left',frameon=False,ncol=2);grid(ax)
save(fig,'q3_final_monthly')

# Q3 10-minute physical curves. Arrays are chronological: slot k starts k/6 h.
# Storage arrays refer to edges; energy orders are multiplied by six for kW.
fig,axes=plt.subplots(3,2,figsize=(10,7.3),sharex=True,sharey='row')
fig.subplots_adjust(left=.08,right=.985,bottom=.13,top=.9,hspace=.48,wspace=.17)
hours=np.arange(144)/6; edges=np.arange(145)/6
handles0=None;handles2=None
for col,date in enumerate(('2025-03-20','2025-09-23')):
    i=int(df.index[df.date==date][0]); day=int(z['days'][i]); ex=z['execution'][i]
    actual=truth[day*144:(day+1)*144]
    q0=z['q0'][i]; final=z['versions'][i,-1]
    ax=axes[0,col]
    l1,=ax.plot(hours,actual[:,0],color='black',lw=1.05,label=' 实际负载')
    l2,=ax.plot(hours,actual[:,1],color=GREEN,lw=1.05,label=' 实际光伏')
    l3,=ax.step(hours,q0*6,where='post',color=GRAY,lw=.9,ls='--',label=' 零点订单')
    l4,=ax.step(hours,final*6,where='post',color=BLUE,lw=1.05,label=' 最终订单')
    handles0=[l1,l2,l3,l4]
    ax.set_title('3 月 20 日' if col==0 else '9 月 23 日',pad=9)
    ax.set_ylim(0,14500);grid(ax)
```

```
    ax=axes[1,col]
    ax.plot(edges,np.r_[ex[0,0],ex[:,1]],color=BLUE,lw=1.2)
    for limit in (1200,10800): ax.axhline(limit,color=GRAY,ls='--',lw=.7)
    ax.set(ylim=(0,11600),yticks=[1200,6000,10800]);grid(ax)
    ax=axes[2,col]
    deficit=np.maximum((actual[:,0]-actual[:,1])/6-final,0)
    f=ax.fill_between(hours,deficit,step='post',color='#d9d9d9',label=' 订单外缺口')
    d,=ax.step(hours,ex[:,7],where='post',color=BLUE,lw=1.1,label=' 电池补充')
    e=ax.bar(hours,ex[:,8],width=1/6,align='edge',color=GOLD,label=' 紧急购电')
    handles2=[f,d,e];ax.set(xlabel='时刻 /
    ↪  h',xlim=(0,24),xticks=[0,6,12,18,24],ylim=(0,1000));grid(ax)
axes[0,0].set_ylabel('功率 / kW');axes[1,0].set_ylabel('储电量 /
↪  kWh');axes[2,0].set_ylabel('每时段补缺电量 / kWh')
fig.legend(handles0,[h.get_label() for h in handles0],loc='upper
↪  center',bbox_to_anchor=(.53,.99),ncol=4,frameon=False)
fig.legend(handles2,['订单外缺口','电池补充','紧急购电'],loc='lower
↪  center',bbox_to_anchor=(.53,.015),ncol=3,frameon=False)
save(fig,'q3_final_days')
```

fig_q3_quantile.py

```
"""Redraw the approved Q3 quantile panel from existing verified results; no experiments."""
from pathlib import Path
import json, hashlib
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT=Path(__file__).resolve().parents[1]
base=ROOT/'reproduced/plots'
(base/'fig').mkdir(parents=True,exist_ok=True)
(base/'scripts').mkdir(exist_ok=True)
source=ROOT/'figure_inputs/derived_values.json'
rows=sorted(json.loads(source.read_text('utf-8'))['q3_quantile_comparison'],key=lambda r:r['q'])
plt.rcParams.update({'font.family':['Times New Roman','SimSun'],'font.size':11,
                     'axes.unicode_minus':False,'pdf.fonttype':42,'ps.fonttype':42})
q=[r['q'] for r in rows]
total=[r['total_cost']/1e4 for r in rows]
ordinary=[r['ordinary_cost']/1e4 for r in rows]
fig,ax=plt.subplots(figsize=(6.8,3.5),layout='constrained')
ax.plot(q,total,'o-',color='#396e96',label=' 总费用',lw=1.15)
ax.plot(q,ordinary,'s--',color='#bf913c',label=' 普通结算',lw=1.05)
ax.fill_between(q,ordinary,total,color='#bf913c',alpha=.16,label=' 紧急购电费')
ax.annotate(f'{total[2]:.2f}',(.6,total[2]),xytext=(0,9),textcoords='offset
↪  points',ha='center',fontsize=10)
ax.set(xticks=q,xlabel=' 误差准备分位',ylabel=' 评价期费用 / 万元',ylim=(1267,1342))
ax.legend(loc='upper center',ncol=3,frameon=False,fontsize=10,columnspacing=1,handlelength=1.5)
ax.grid(axis='y',color='#dddddd',linewidth=.5)
ax.set_axisbelow(True)
ax.spines[['top','right']].set_visible(False)
for ext in ('pdf','png'):
    kw={'metadata':{'CreationDate':None,'ModDate':None}} if ext=='pdf' else {'dpi':300}
    fig.savefig(base/f'fig/q3_final_quantile.{ext}',bbox_inches='tight',**kw)
plt.close(fig)
(base/'scripts/q3_quantile_manifest.json').write_text(json.dumps({
    'source':'scripts/derived_values.json','source_sha256':hashlib.sha256(source.read_bytes()).hexdi
    ↪  gest(),
```

```
    'data_key':'q3_quantile_comparison','script':'scripts/draw_q3_quantile.py',
    'outputs':['fig/q3_final_quantile.pdf','fig/q3_final_quantile.png'],
    'scope':'Existing result visualization only; no new modeling or validation.'},ensure_ascii=False,indent=2),'utf-8')
print('Q3 quantile PDF and PNG generated from existing result data.')
```

framework.tex

```
\documentclass{article}
\usepackage[paperwidth=198mm,paperheight=154mm,margin=3mm]{geometry}
\usepackage[UTF8]{ctex}
\usepackage{fontspec,amsmath,tikz}
\setmainfont{Times New Roman}
\setCJKmainfont{SimSun}[BoldFont=SimHei]
\usetikzlibrary{arrows.meta}
\pagestyle{empty}
\setlength{\parindent}{0pt}
\linespread{1.0}\selectfont
\begin{document}
\centering
\begin{tikzpicture}[x=1cm,y=1cm,
 every node/.style={font=\fontsize{10}{13}\selectfont,align=center,inner sep=3pt,text=black},
 panel/.style={draw=black,line width=.9pt,fill=white},
 box/.style={draw=black,line width=.6pt,fill=white,minimum height=1cm,text width=3.7cm,font=\fontsize{10}{13}\selectfont\bfseries},
 core/.style={box,line width=1.15pt},
 arr/.style={-{Stealth[length=1.7mm,width=1.3mm]},line width=.7pt},
 head/.style={font=\fontsize{11}{14}\selectfont\bfseries,anchor=west},
 note/.style={font=\fontsize{9}{12}\selectfont}]
\path[use as bounding box] (0,.3) rectangle (19.1,14.9);

% Labels remaining after the five requested screenshot deletions are centered.
\draw[panel] (0,12.55) rectangle (19.1,14.9);
\draw[panel] (0,8.95) rectangle (19.1,11.85);
\draw[panel] (0,5.35) rectangle (19.1,8.25);
\draw[panel] (0,.3) rectangle (19.1,4.65);
\foreach \yt in {14.9,11.85,8.25,4.65}{\draw[line width=.5pt] (0,\yt-.72)--(19.1,\yt-.72);}

\node[head] at (.27,14.53) {问题一\quad 全时段电量流线性规划};
\node[box] (a1) at (2.35,13.37) {代表日供需与电价};
\node[box] (a2) at (7.12,13.37) {用电缺口与光伏余量};
\node[core] (a3) at (11.91,13.37) {全时段线性规划};
\node[box] (a4) at (16.7,13.37) {当天最低购电费};
\foreach \from/\to in {a1/a2,a2/a3,a3/a4}{\draw[arr] (\from.east)--(\to.west);}

\draw[arr] (9.55,12.55)--(9.55,11.85);
\node[note,fill=white,anchor=east] at (9.30,12.20) {继承供需与储能约束};
\node[note,fill=white,anchor=west] at (9.80,12.20) {新增：供需未知、跨日运行与紧急购电};

\node[head] at (.27,11.48) {问题二\quad 历史分项预测—近期加权准备—跨日线性规划};
\node[box] (b1) at (2.35,10.30) {零点可用信息};
\node[box] (b2) at (7.12,10.30) {负载、光伏分别预测};
\node[box] (b3) at (11.91,10.30) {近期加权误差准备};
\node[core] (b4) at (16.7,10.30) {跨日线性规划};
\foreach \from/\to in {b1/b2,b2/b3,b3/b4}{\draw[arr] (\from.east)--(\to.west);}
\node[draw,line width=.5pt,text width=17.8cm,minimum height=.57cm,font=\fontsize{10}{13}\selectfont\bfseries] (bex) at (9.55,9.40) {
```

```
在供需不确定时，降低普通购电与紧急购电的总费用};
\draw[arr] (b4.east)--(18.9,10.30)--(18.9,9.40)--(bex.east);

\draw[arr] (9.55,8.95)--(9.55,8.25);
\node[note,fill=white,anchor=east] at (9.30,8.60) {继承负载预测、误差准备与跨日储能};
\node[note,fill=white,anchor=west] at (9.80,8.60) {新增：光伏预报更新、有代价的改单};

\node[head] at (.27,7.88) {问题三\quad 融合预测与日内校正滚动规划};
\node[box] (c1) at (2.35,6.70) {0、6、12、18 点新预报};
\node[box] (c2) at (7.12,6.70) {负载校正与光伏融合};
\node[box] (c3) at (11.91,6.70) {近期误差准备};
\node[core] (c4) at (16.7,6.70) {日内滚动线性规划};
\foreach \from/\to in {c1/c2,c2/c3,c3/c4}{\draw[arr] (\from.east)--(\to.west);}
\node[draw,line width=.5pt,text width=17.8cm,minimum height=.57cm,font=\fontsize{10}{13}\selectfont\bfseries] (cex) at (9.55,5.80) {
降低普通、调整与紧急购电的总费用，并评价日内光伏预报的价值};
\draw[arr] (c4.east)--(18.9,6.70)--(18.9,5.80)--(cex.east);

\draw[arr] (9.55,5.35)--(9.55,4.65);
\node[note,fill=white,anchor=east] at (9.30,5.0) {分别扩展问题二、问题三的调度结构};
\node[note,fill=white,anchor=west] at (9.80,5.0) {新增：未来电价未知};

\node[head,font=\fontsize{10.5}{14}\selectfont\bfseries] at (.27,4.28) {问题四\quad 梯度提升树—零点线性规划 / 岭回归—日内调整线性规划};
\node[anchor=west,font=\fontsize{10}{13}\selectfont\bfseries] at (.27,3.59) {问题四-2：沿用问题二的决策权限};
\node[box] (d1) at (2.35,2.86) {零点以前的历史信息};
\node[core] (d2) at (7.12,2.86) {梯度提升树预测电价};
\node[core] (d3) at (11.91,2.86) {零点线性规划};
\node[box] (d4) at (16.7,2.86) {当天固定购电计划};
\foreach \from/\to in {d1/d2,d2/d3,d3/d4}{\draw[arr] (\from.east)--(\to.west);}
\draw[dashed,line width=.5pt] (.27,2.17)--(18.83,2.17);
\node[anchor=west,font=\fontsize{10}{13}\selectfont\bfseries] at (.27,1.88) {问题四-3：沿用问题三的决策权限};
\node[box] (e1) at (2.35,1.15) {历史与当日新增信息};
\node[core] (e2) at (7.12,1.15) {岭回归预测电价};
\node[core] (e3) at (11.91,1.15) {日内调整线性规划};
\node[box] (e4) at (16.7,1.15) {零点计划与日内调整};
\foreach \from/\to in {e1/e2,e2/e3,e3/e4}{\draw[arr] (\from.east)--(\to.west);}
\end{tikzpicture}
\end{document}

```

prepare_inputs.py

```
"""Rebuild numerical inputs from the included, value-preserving CSV exports."""
from pathlib import Path
import numpy as np
import pandas as pd
ROOT=Path(__file__).resolve().parents[1]
def main():
 date=pd.Timestamp('2025-01-01');read=lambda n:pd.read_csv(ROOT/'data'/f'{n}.csv.gz',float_precision='round_trip')
 ref=read('day_reference');ref=ref.assign(t=ref.source_minute%1440).sort_values('t')
 a=read('actual_power');t=pd.to_datetime(a.source_date)+pd.to_timedelta(a.source_minute,unit='m')
 ix=((t-date).dt.total_seconds()/600).astype(int).to_numpy();assert np.array_equal(ix,np.arange(1,52561))
```

```
 actual=np.full((52561,2),np.nan);actual[ix]=a[['load_kw','pv_actual_kw']].to_numpy()
 f=read('pv_forecasts');release=np.arange(1460)*36;targets=release[:,None]+np.arange(1,25)[None,:]*6
 rr=((pd.to_datetime(f.release_datetime)-date).dt.total_seconds()/600).astype(int).to_numpy()
 tt=((pd.to_datetime(f.target_datetime)-date).dt.total_seconds()/600).astype(int).to_numpy()
 assert np.array_equal(rr,np.repeat(release,24)) and np.array_equal(tt,targets.ravel())
 p=read('actual_price');p['day']=(pd.to_datetime(p.source_date)-date).dt.days;p['slot']=p.source_min
 ↪ ute.astype(int)%1440//10
 prices=p.pivot(index='day',columns='slot',values='price_yuan_per_kwh').reindex(index=range(365),col
 ↪ umns=range(144)).to_numpy()
 data=dict(actual=actual,reference=ref[['load_kw','pv_forecast_kw']].to_numpy(),price=ref.price_yuan
 ↪ _per_kwh.to_numpy(),F=f.pv_forecast_kw.to_numpy().reshape(1460,24),release=release,targets=targ
 ↪ ets,actual_prices=prices)
 saved=np.load(ROOT/'data/inputs.npz')
 for k,v in data.items():np.testing.assert_array_equal(v,saved[k],err_msg=k)
 out=ROOT/'reproduced';out.mkdir(exist_ok=True);np.savez_compressed(out/'inputs.npz',**data)
 print('Prepared arrays match all saved inputs exactly.')
if __name__=='__main__':main()
```

prepare_source_tables.py

```
"""Execute approved Data-plan-v1. Copy source values; no interval interpolation.

python code/prepare_source_tables.py
Use --output <new project-relative directory> for non-overwriting reproduction.
"""
import argparse
import csv
from datetime import date, datetime, time, timedelta
import hashlib
import json
from pathlib import Path
import re
import sys
import xml.etree.ElementTree as ET
import zipfile

import openpyxl
from openpyxl.utils import get_column_letter

ROOT = Path(__file__).resolve().parents[1]
NS = {'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
REL = '{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id'
SOURCES = {
    'annex1': ('data/raw/C 题/附件/附件 1.xlsx', 'Sheet1'),
    'annex2_load': ('data/raw/C 题/附件/附件 2.xlsx', ' 小区负载'),
    'annex2_pv': ('data/raw/C 题/附件/附件 2.xlsx', ' 光伏发电实际功率'),
    'annex3': ('data/raw/C 题/附件/附件 3.xlsx', 'Sheet1'),
    'annex4': ('data/raw/C 题/附件/附件 4.xlsx', 'Sheet1'),
}


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def label(value):
```

```
    return value.isoformat() if isinstance(value,(date,datetime,time)) else str(value or '')


def parse_date(value):
    if isinstance(value,datetime):
        return value.date()
    if isinstance(value,date):
        return value
    return date(*map(int,re.split(r'[-/.]',value.strip())))


def minutes(value):
    if isinstance(value,time):
        assert value.second == 0 and value.microsecond == 0
        return value.hour*60+value.minute
    m=re.fullmatch(r'(\d{1,2}):(\d{1,2})(\+1)?',value.strip())
    assert m, value
    return int(m[1])*60+int(m[2])+(1440 if m[3] else 0)


def source(sid):
    path,sheet=SOURCES[sid]
    wb=openpyxl.load_workbook(ROOT/path,read_only=True,data_only=False)
    ws=wb[sheet]
    rows=list(ws.iter_rows(values_only=True))
    wb.close()
    with zipfile.ZipFile(ROOT/path) as z:
        tree=ET.fromstring(z.read('xl/workbook.xml'))
        rid=next(s for s in tree.find('s:sheets',NS) if s.attrib['name']==sheet).attrib[REL]
        target=next(r.attrib['Target'] for r in ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))
        ↪  if r.attrib['Id']==rid)
        target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
        values={}
        for cell in ET.fromstring(z.read(target)).findall('.//s:sheetData/s:row/s:c',NS):
            v=cell.find('s:v',NS)
            if v is not None and cell.attrib.get('t','n')=='n':
                values[cell.attrib['r']]=v.text
    return rows,values


def write_csv(out,name,fields,records):
    path=out/name
    with path.open('x',encoding='utf-8-sig',newline='') as stream:
        writer=csv.DictWriter(stream,fieldnames=fields)
        writer.writeheader()
        writer.writerows(records)
    return {'rows':len(records),'columns':fields,'sha256':sha(path)}


def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--output',default='reproduced/source_tables')
    args=parser.parse_args()
    out=(ROOT/args.output).resolve()
    if ROOT not in out.parents or (ROOT/'data/raw').resolve() in out.parents or
    ↪  out==(ROOT/'data/raw').resolve():
        raise ValueError('Output must be a new project directory outside raw')
    if out.exists():
```

```
        raise FileExistsError(f'Will not overwrite: {out}')
    expected=json.loads((ROOT/'data/raw_hashes.json').read_text(encoding='utf-8'))
    if any(sha(ROOT/p)!=h for p,h in expected.items()):
        raise RuntimeError('Source hashes differ from approved audit')
    loaded={sid:source(sid) for sid in SOURCES}
    tables={}
    rows,raw=loaded['annex1']
    records=[]
    for r in range(2,len(rows)+1):
        t=rows[r-1][0]
        records.append(dict(slot_index=r-1,source_time_label=label(t),source_time_type=type(t).__nam
        ↪ e__,source_minute=minutes(t),price_yuan_per_kwh=raw[f'B{r}'],load_kw=raw[f'C{r}'],pv_for
        ↪ ecast_kw=raw[f'D{r}'],source_id='annex1',time_cell=f'A{r}',price_cell=f'B{r}',load_cell=
        ↪ f'C{r}',pv_forecast_cell=f'D{r}'))
    tables['day_reference.csv']=records

    def matrix(sid):
        rows,raw=loaded[sid]
        result={}
        for r in range(2,len(rows)+1):
            d=parse_date(rows[r-1][0])
            for c in range(2,len(rows[0])+1):
                t=rows[0][c-1]
                key=(d.isoformat(),minutes(t))
                if key in result:
                    raise ValueError(f'Duplicate source key: {sid} {key}')
                result[key]=dict(source_date=d.isoformat(),source_date_label=label(rows[r-1][0]),sou
                ↪ rce_date_type=type(rows[r-1][0]).__name__,source_time_label=label(t),source_time
                ↪ _type=type(t).__name__,source_minute=minutes(t),value=raw[f'{get_column_letter(c
                ↪ )}{r}'],source_id=sid,date_cell=f'A{r}',time_cell=f'{get_column_letter(c)}1',val
                ↪ ue_cell=f'{get_column_letter(c)}{r}')
        return result
    load,pv,price=(matrix(s) for s in ['annex2_load','annex2_pv','annex4'])
    if set(load)!=set(pv) or set(load)!=set(price):
        raise ValueError('Actual axes do not align exactly')
    common=['source_date','source_date_label','source_date_type','source_time_label','source_time_ty
    ↪ pe','source_minute']
    records=[]
    for key in sorted(load):
        a,b=load[key],pv[key]
        assert all(a[k]==b[k] for k in common)
        record={k:a[k] for k in common}
        record.update(load_kw=a['value'],pv_actual_kw=b['value'])
        for prefix,item in [('load',a),('pv',b)]:
            record.update({f'{prefix}_{k}':item[k] for k in
            ↪ ['source_id','date_cell','time_cell','value_cell']})
        records.append(record)
    tables['actual_power.csv']=records
    records=[]
    for key in sorted(price):
        a=price[key]
        record={k:a[k] for k in common}
        record.update(price_yuan_per_kwh=a['value'])
        record.update({k:a[k] for k in ['source_id','date_cell','time_cell','value_cell']})
        records.append(record)
    tables['actual_price.csv']=records

```

```
    actual_labels={datetime.fromisoformat(d)+timedelta(minutes=m) for d,m in load}
    rows,raw=loaded['annex3']
    records=[]
    inherited_rows=0
    for group_row in range(2,len(rows)+1,4):
        group_date=rows[group_row-1][0]
        d=parse_date(group_date)
        for offset in range(4):
            r=group_row+offset
            release_value=rows[r-1][1]
            release=datetime.combine(d,time())+timedelta(minutes=minutes(release_value))
            inherited=int(rows[r-1][0] in (None,''))
            inherited_rows+=inherited
            for lead in range(1,25):
                c=lead+2
                target=release+timedelta(hours=lead)
                records.append(dict(release_datetime=release.isoformat(),target_datetime=target.isof
                ↪  ormat(),lead_hours=lead,pv_forecast_kw=raw[f'{get_column_letter(c)}{r}'],source_
                ↪  date_label=label(group_date),source_date_type=type(group_date).__name__,source_r
                ↪  ow_date_label=label(rows[r-1][0]),source_row_date_type=type(rows[r-1][0]).__name
                ↪  __,source_release_label=label(release_value),source_release_type=type(release_va
                ↪  lue).__name__,date_inherited=inherited,has_actual_target_label=int(target in
                ↪  actual_labels),source_id='annex3',group_date_cell=f'A{group_row}',row_date_cell=
                ↪  f'A{r}',release_cell=f'B{r}',value_cell=f'{get_column_letter(c)}{r}'))
    tables['pv_forecasts.csv']=records
    expected_rows={'day_reference.csv':144,'actual_power.csv':52560,'actual_price.csv':52560,'pv_for
    ↪  ecasts.csv':35040}
    assert {k:len(v) for k,v in tables.items()}==expected_rows
    out.mkdir(parents=True,exist_ok=False)
    outputs={name:write_csv(out,name,list(records[0]),records) for name,records in tables.items()}
    readme='''# 整理后的来源数据（prepared-v1）

按已批准的最小格式整理方案生成，当前是待用户确认的候选版本。
保留全部原始数值、来源位置和发布版本，没有训练、插值、能量换算或填写购电结果。

| 文件 | 每行表示什么 | 行数 | 数值单位 |
|---|---|---:|---|
| day_reference.csv | 附件 1 代表日的一个原始时间标签 | 144 | 功率 kW；电价元/kWh |
| actual_power.csv | 一个来源日期和原时间标签的实际负载及光伏 | 52560 | kW |
| actual_price.csv | 一个来源日期和原时间标签的实际价格 | 52560 | 元/kWh |
| pv_forecasts.csv | 一次发布针对一个未来整点的光伏预报 | 35040 | kW |

## 时间字段

source_date 保留原表归属日；代表日没有日历日期，不假造日期。
source_minute 为距离归属日零点的分钟数：0:10 对应 10，0:00+1 对应 1440。
source_time_label保留原单元格值的文字表示：Excel时间转ISO文字，原文本不改；source_time_type保留原类型。
这只是来源标签，不说明功率代表前一段、后一段或平均值，不能直接把它当作已确认的购电区间。
CSV 保留日期和时间为明确文字、分钟数为整数。不要覆盖 CSV 后再声称原值未变。

## 预报字段

release_datetime 是发布时刻；lead_hours 为 1—24；target_datetime 等于发布时刻加提前小时数。
同一目标时刻的不同发布版本全部保留，不能按目标时刻去重或用较晚发布值覆盖较早发布。
date_inherited=1 表示原行日期空白，来自同组首行；原行日期及组首单元格都保留。
has_actual_target_label 只表示目标标签是否在附件 2 中存在，不代表当时可提前获知其实际值。
全年最后实际标签是 2025-12-31 的 0:00+1，即 2026-01-01 00:00；36 条预报晚于此时刻，仅标注而未删除。

```

```
## 信息身份与单位

actual_power 是实际观测；actual_price 是实际价格，在 Q4 中不可提前知道未来部分。
day_reference 来自附件 1，其负载/预测用于 Q1 和已确认的 Q2 首日启动参考；其电价用于固定日曲线场景。
pv_forecasts 来自附件 3，仅在相应发布时刻起可用，不能用于未经授权的 Q2/Q4-2 预报输入。
CSV 本身不强制这些权限，后续研究代码还需落实时间切片。
功率仍为 kW，价格仍为元/kWh；没有得到区间能量 kWh，未填补数值、裁剪、平滑、标准化或插值。

## 数值及来源

数值列直接保存 XLSX 内部的数值文本，没有按显示小数位四舍五入；重读时按数值解析。
source_id 在 manifest.json 的 sources 中映射到原文件与工作表。
所有 *_cell 列均为原工作表单元格地址；实际功率的 load_/pv_ 前缀分别指两张原表。
source_date_label/type 保留原日期值表示及类型；source_row_date_label/type 保留预报当前行的原日期空白。
source_release_label/type 保留预报发布时间的原值表示及类型。
slot_index 仅是附件 1 原行顺序索引，不是新定义的物理区间。

manifest.json 记录文件哈希、来源身份、行数、字段、生成脚本和授权。
逐值核对结果另存项目 audit/prepared_v1_verification.json，不是人工核验或数据冻结。

## 复现

在仓库根目录使用 Python 运行：
python code/prepare_source_tables.py --output tmp/prepared-v1-reproduction
只允许新目录；脚本拒绝覆盖已有输出或写入原始材料目录。
核验命令：python -X utf8 projects/2026-C/scripts/verify_prepared_tables.py
'''
    (out/'README.md').write_text(readme,encoding='utf-8')
    if any(sha(ROOT/p)!=h for p,h in expected.items()):
        raise RuntimeError('Original files changed during preparation')
    manifest={'version':'prepared-v1','status':'candidate_pending_user_confirmation','plan_version':'Data-plan-v1','authorization':'DATA-PREP-012','created_at':datetime.now().astimezone().isoformat(),'python':sys.version,'openpyxl':openpyxl.__version__,'script':'scripts/prepare_source_tables.py','script_sha256':sha(Path(__file__)),'raw_sha256':expected,'raw_unchanged_after_preparation':True,'sources':{sid:{'path':path,'sheet':sheet} for sid,(path,sheet) in SOURCES.items()},'outputs':outputs,'readme_sha256':sha(out/'README.md'),'transformations':{'original_numeric_values_changed':0,'original_numeric_cells':193152,'total_output_records':sum(expected_rows.values()),'forecast_rows_with_inherited_group_date':inherited_rows,'forecast_records_without_actual_target_label':sum(not r['has_actual_target_label'] for r in tables['pv_forecasts.csv']),'numeric_text':'copied verbatim from original XLSX numeric value elements','physical_interval_mapping':'not selected','hourly_interpolation':'not performed','power_to_energy':'not performed'},'information_roles':{'day_reference.csv':'attachment1_reference_not_Q2_future_actuals','actual_power.csv':'actual_observation_not_future_information','actual_price.csv':'Q4_actual_price_not_known_ahead','pv_forecasts.csv':'available_only_at_or_after_release_in_authorized_branches'}}
    (out/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
    print(json.dumps({'output':str(out),'rows':expected_rows,'numeric_values_changed':0,'inherited_date_rows':inherited_rows,'forecast_outside_actual_labels':manifest['transformations']['forecast_records_without_actual_target_label']},ensure_ascii=False))


if __name__=='__main__':
    main()
```

q1.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
def arrays(inputs, config):
    dt=config['dt_hours']
    l=np.array([r['load_kw']*dt for r in inputs]); v=np.array([r['pv_kw']*dt for r in inputs])
    p=np.array([r['price'] for r in inputs])
    return l, v, p, np.maximum(l-v,0), np.maximum(v-l,0)

def terminal_bands(net, cfg):
    n=len(net); low=np.empty(n+1); high=np.empty(n+1)
    low[n]=high[n]=cfg['terminal_kwh']
    up=cfg['eta_charge']*cfg['external_charge_kw']*cfg['dt_hours']
    down=np.minimum(net,cfg['external_discharge_kw']*cfg['dt_hours'])/cfg['eta_discharge']
    for t in range(n-1,-1,-1):
        low[t]=max(cfg['min_kwh'],low[t+1]-up)
        high[t]=min(cfg['max_kwh'],high[t+1]+down[t])
    return low,high,up,down

def rule(inputs, cfg, low_q, high_q, target_fraction, reserve):
    l,v,p,net,excess=arrays(inputs,cfg); n=len(p)
    low_price=float(np.quantile(p,low_q)); high_price=float(np.quantile(p,high_q))
    cap_c=cfg['external_charge_kw']*cfg['dt_hours'];
    cap_d=cfg['external_discharge_kw']*cfg['dt_hours']
    eta_c=cfg['eta_charge']; eta_d=cfg['eta_discharge']; smin=cfg['min_kwh']; smax=cfg['max_kwh']
    target=smin+target_fraction*(smax-smin)
    lo,hi,up,down=terminal_bands(net,cfg)
    state=cfg['initial_kwh']; rows=[]
```

```
    for t in range(n):
        desired=0.; cap=target; reserve_amount=0.
        reason='idle'
        if excess[t]>1e-10:
            desired=min(eta_c*min(excess[t],cap_c),smax-state)
            reason='pv_charge'
        elif p[t]>=high_price:
            desired=-min(down[t],state-smin)
            reason='peak_discharge'
        elif p[t]<=low_price:
            future_prices=p[t+1:][net[t+1:]>1e-10]
            economical=len(future_prices)>0 and p[t]<eta_c*eta_d*float(future_prices.max())
            if reserve:
                future=np.flatnonzero(excess[t+1:]>1e-10)
                if len(future):
                    a=t+1+int(future[0]); b=a
                    while b<n and excess[b]>1e-10: b+=1
                    incoming=eta_c*float(np.minimum(excess[a:b],cap_c).sum())
                    release=float(down[t+1:a][p[t+1:a]>=high_price].sum())
                    reserve_amount=min(smax-smin,max(0.,incoming-release))
                    cap=min(target,max(smin,smax-reserve_amount))
            if economical:
                desired=min(up,max(0.,cap-state)); reason='low_price_charge'
        allowed_low=max(-down[t],lo[t+1]-state,smin-state)
        allowed_high=min(up,hi[t+1]-state,smax-state)
        assert allowed_low<=allowed_high+1e-7, (t,state,allowed_low,allowed_high)
        actual=float(np.clip(desired,allowed_low,allowed_high))
        c=max(actual,0.)/eta_c; discharge=max(-actual,0.)*eta_d
        cpv=min(c,excess[t]); cg=c-cpv
        before=state; state+=actual
        rows.append(dict(grid_charge=cg,pv_charge=cpv,discharge=discharge,state_start=before,state_e
        ↪ nd=state,
            action_reason=reason,desired_delta=desired,actual_delta=actual,
            terminal_projection=int(abs(actual-desired)>1e-7),
            ↪ reserve_kwh=reserve_amount,grid_charge_target=cap))
    return rows,dict(low_quantile=low_q,high_quantile=high_q,target_fraction=target_fraction,
        low_price=low_price,high_price=high_price,target_kwh=target)

def solve_lp(inputs,cfg):
    l,v,p,net,excess=arrays(inputs,cfg); n=len(p)
    # x = [grid_charge, pv_charge, load-side discharge, end storage]
    objective=np.r_[p,np.zeros(n),-p,np.zeros(n)]
    aeq=np.zeros((n,4*n)); beq=np.zeros(n)
    for t in range(n):
        aeq[t,t]=-cfg['eta_charge']; aeq[t,n+t]=-cfg['eta_charge']
        aeq[t,2*n+t]=1/cfg['eta_discharge']; aeq[t,3*n+t]=1
        if t: aeq[t,3*n+t-1]=-1
        else: beq[t]=cfg['initial_kwh']
    aub=np.zeros((n,4*n))
    for t in range(n): aub[t,t]=aub[t,n+t]=1
    cc=cfg['external_charge_kw']*cfg['dt_hours']; dc=cfg['external_discharge_kw']*cfg['dt_hours']
    lower=np.r_[np.zeros(3*n),np.full(n,cfg['min_kwh'])]
    upper=np.r_[np.full(n,cc),np.minimum(excess,cc),np.minimum(net,dc),np.full(n,cfg['max_kwh'])]
    lower[-1]=upper[-1]=cfg['terminal_kwh']
    before=time.perf_counter()
    result=linprog(objective,A_ub=aub,b_ub=np.full(n,cc),A_eq=aeq,b_eq=beq,
        bounds=list(zip(lower,upper)),method='highs',options=dict(time_limit=cfg['lp_time_limit_seco
        ↪ nds'],
```

```
            primal_feasibility_tolerance=cfg['lp_feasibility_tolerance'],
            dual_feasibility_tolerance=cfg['lp_feasibility_tolerance']))
    elapsed=time.perf_counter()-before
    assert result.success, result.message
    x=result.x; base=float(p@net)
    dual=float(beq@result.eqlin.marginals+np.full(n,cc)@result.ineqlin.marginals+
        lower@result.lower.marginals+upper@result.upper.marginals+base)
    cert=dict(status=int(result.status),message=result.message,method='HiGHS
    ↪ LP',iterations=int(result.nit),
        time_seconds=elapsed,primal_cost=float(result.fun+base),dual_cost=dual,
        primal_dual_gap=float(result.fun+base-dual),objective=objective.tolist(),x=x.tolist(),
        equality_marginals=result.eqlin.marginals.tolist(),inequality_marginals=result.ineqlin.margi
        ↪ nals.tolist(),
        lower_marginals=result.lower.marginals.tolist(),upper_marginals=result.upper.marginals.tolis
        ↪ t())
    rows=[]
    for t in range(n):
        s0=cfg['initial_kwh'] if t==0 else x[3*n+t-1]
        rows.append(dict(grid_charge=float(x[t]),pv_charge=float(x[n+t]),discharge=float(x[2*n+t]),
            state_start=float(s0),state_end=float(x[3*n+t]),action_reason='whole_day_optimization',
            desired_delta=float(x[3*n+t]-s0),actual_delta=float(x[3*n+t]-s0),
            terminal_projection=0,reserve_kwh=0.,grid_charge_target=0.))
    return rows,cert

def schedule(inputs, cfg, actions):
    l,v,p,net,excess=arrays(inputs,cfg); records=[]
    for t,(r,a) in enumerate(zip(inputs,actions)):
        charge=a['grid_charge']+a['pv_charge']; q=net[t]-a['discharge']+a['grid_charge']
        rec=dict(r); rec.update(a)
        rec.update(load_kwh=float(l[t]),pv_kwh=float(v[t]),direct_pv_kwh=float(min(l[t],v[t])),
            grid_to_load_kwh=float(net[t]-a['discharge']),purchase_kwh=float(q),charge_kwh=float(cha
            ↪ rge),
            unused_pv_kwh=float(excess[t]-a['pv_charge']),
            loss_kwh=float((1-cfg['eta_charge'])*charge+(1/cfg['eta_discharge']-1)*a['discharge']),
            cost_yuan=float(p[t]*q))
        records.append(rec)
    return records

def metrics(rows):
    totals={key:float(sum(r[key] for r in rows)) for key in
        ['cost_yuan','purchase_kwh','charge_kwh','discharge','pv_charge','grid_charge','unused_pv_kw
        ↪ h','loss_kwh']}
    states=[rows[0]['state_start']]+[r['state_end'] for r in rows]
    totals.update(start_kwh=states[0],end_kwh=states[-1],min_kwh=min(states),max_kwh=max(states),
        max_charge_kw=max(r['charge_kwh']*6 for r in rows),max_discharge_kw=max(r['discharge']*6 for
        ↪ r in rows),
        simultaneous_slots=sum(r['charge_kwh']>1e-5 and r['discharge']>1e-5 for r in rows),
        terminal_projection_slots=sum(r['terminal_projection'] for r in rows))
    return totals
```

q2.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
```

```
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
class Data:
    def __init__(self,source):
        self.a={"load":source["actual"][:N,0].reshape(365,144),"pv":source["actual"][:N,1].reshape(365,144)}
        self.ref={"load":source["reference"][:,0],"pv":source["reference"][:,1]}
        self.price=source["price"]

    def refmean(self,var,d,h,mode,n):
        # Only target-day calendar is future-known; all observed values have dates < d.
        if mode=='weekday':idx=[d+h-7*j for j in range(1,n+1) if 0<=d+h-7*j<d]
        else:idx=list(range(max(0,d-n),d))
        fallback=False
        if not idx:idx=list(range(max(0,d-7),d));fallback=True
        assert idx and max(idx)<d
        z=self.a[var][idx];count=np.isfinite(z).sum(axis=0)
        ans=np.divide(np.nansum(z,axis=0),count,out=self.ref[var].copy(),where=count>0)
        meta={'variable':var,'decision_date':daystr(d),'horizon_day':h+1,'mode':mode,'requested_n':n,
              'history_dates':[daystr(i) for i in idx],'weight_per_available_value':'1/count_at_slot',
              'count_min':int(count.min()),'count_max':int(count.max()),
              'calendar_fallback':fallback,'reference_only_slots':np.flatnonzero(count==0).tolist(),
              'latest_history_interval_end':daystr(max(idx)+1)+' 00:00'}
        return ans,meta

    def history(self,d,w,pv_days):
        L=[];V=[];weights=[]
        for h in [0,1]:
            l,lm=self.refmean('load',d,h,'weekday',w)
            v,vm=self.refmean('pv',d,h,'recent',pv_days)
            L.append(l);V.append(v);weights.extend([lm,vm])
        return np.concatenate(L),np.concatenate(V),weights
```

```
60
61      def truth(self,d,initialization=False):
62          l=self.a['load'][d].copy();v=self.a['pv'][d].copy()
63          if d==0 and initialization:l[0]=self.ref['load'][0];v[0]=self.ref['pv'][0]
64          assert np.isfinite(l).all() and np.isfinite(v).all()
65          return l,v
66
67      def features(self,d,h,var):
68          k=np.arange(144);t=2*np.pi*k/144;target=pd.Timestamp(daystr(d+h));dow=target.dayofweek
69          cols=[np.sin(t),np.cos(t),np.sin(2*t),np.cos(2*t)]
70          names=['time_sin1','time_cos1','time_sin2','time_cos2']
71          for i in range(7):cols.append(np.full(144,int(dow==i)));names.append('weekday_'+str(i))
72          cols.extend([np.full(144,h+1),np.full(144,np.sin(2*np.pi*target.dayofyear/365)),
73                       np.full(144,np.cos(2*np.pi*target.dayofyear/365))])
74          names.extend(['horizon_day','year_sin','year_cos'])
75          for mode,n,name in [('recent',1,'recent_slot_last'),('recent',3,'recent_slot_mean3'),
76                              ('recent',7,'recent_slot_mean7'),('weekday',1,'matching_week1'),('weekda
                                ↪ y',4,'matching_week4')]:
77              value,_=self.refmean(var,d,h,mode,n);cols.append(value);names.append(name)
78          cols.extend([np.full(144,np.nanmean(self.a[var][d-1])),
79                       np.full(144,np.nanmean(self.a[var][max(0,d-7):d]))])
80          names.extend(['last_daily_level','recent7_daily_level'])
81          x=np.column_stack(cols);assert np.isfinite(x).all()
82          return x,names
83
84  class Forecasts:
85      def __init__(self,data,out,event,guard):
86          self.data=data;self.out=out;self.event=event;self.guard=guard;self.models={};self.cache={};s
            ↪ elf.fitcount=0
87          self.X={};self.y={};self.origins=[];self.targets=[]
88          for var in ['load','pv']:
89              xs=[];ys=[];orig=[];target=[]
90              for d in range(2,365):
91                  for h in [0,1]:
92                      if d+h>=365:continue
93                      x,self.names=data.features(d,h,var);xs.append(x);ys.append(data.a[var][d+h])
94                      orig.extend([d]*144);target.extend([d+h]*144)
95              self.X[var]=np.vstack(xs);self.y[var]=np.concatenate(ys)
96              self.origins=np.array(orig);self.targets=np.array(target)
97          np.savez_compressed(out/'feature_bank.npz',load_X=self.X['load'],pv_X=self.X['pv'],
98                              load_y=self.y['load'],pv_y=self.y['pv'],origins=self.origins,targets=sel
                                ↪ f.targets)
99          dump(out/'feature_names.json',self.names)
100         (out/'models').mkdir()
101
102     def fit(self,kind,param,d):
103         fitid=f'{kind}-{param}-{daystr(d)}';modelpair={}
104         eligible=(self.origins>=max(2,d-56))&(self.targets<d)
105         assert eligible.any() and int(self.targets[eligible].max())<d
106         for var in ['load','pv']:
107             self.guard();x=self.X[var][eligible];y=self.y[var][eligible];start=time.perf_counter()
108             scaler=None
109             if kind=='T2':
110                 scaler=StandardScaler();xt=scaler.fit_transform(x);model=Ridge(alpha=float(param));m
                    ↪ odel.fit(xt,y)
111             else:
112                 model=HistGradientBoostingRegressor(max_depth=int(param),max_iter=200,learning_rate=
                    ↪ .05,
```

```
                    min_samples_leaf=48,l2_regularization=1.,max_bins=63,early_stopping=False,random_state=2026)
                model.fit(x,y)
            elapsed=time.perf_counter()-start
            if elapsed>60:raise RuntimeError('Single fit budget exceeded')
            self.fitcount+=1
            joblib.dump({'model':model,'scaler':scaler,'feature_names':self.names,'cutoff':daystr(d)},
                        self.out/'models'/f'{fitid}-{var}.joblib',compress=3)
            record={'fit_id':fitid,'variable':var,'cutoff':daystr(d),'n_rows':int(eligible.sum()),
                    'history_origin_min':daystr(self.origins[eligible].min()),
                    'latest_target_end':daystr(self.targets[eligible].max()+1)+'00:00','seconds':elapsed,
                    'sample_weight':'uniform1','hyperparameter':param,'model':kind}
            if kind=='T2':
                unscaled=model.coef_/scaler.scale_
                record.update(coefficients_standardized=model.coef_.tolist(),coefficients_original=unscaled.tolist(),
                    feature_means=scaler.mean_.tolist(),feature_scales=scaler.scale_.tolist(),
                    intercept_original=float(model.intercept_-np.dot(unscaled,scaler.mean_)),
                    intercept_standardized=float(model.intercept_))
            else:
                counts=np.zeros(len(self.names),dtype=int)
                for group in model._predictors:
                    for tree in group:
                        nodes=tree.nodes
                        for z in nodes[nodes['is_leaf']==0]:counts[int(z['feature_idx'])]+=1
                record.update(learning_rate=.05,iterations=int(model.n_iter_),split_feature_counts=counts.tolist(),
                              split_counts_are_not_weights_or_causal_importance=True)
            self.event('fit_log',record);modelpair[var]=(model,scaler)
        self.models[(kind,param)]=(d,fitid,modelpair)
        return self.models[(kind,param)]

    def get(self,kind,param,d,phase_start):
        key=(kind,str(param),d,phase_start)
        if key in self.cache:return self.cache[key]
        if kind=='T1':
            w,p=param;l,v,weights=self.data.history(d,w,p);fitid='equal-history'
            for row in weights:self.event('history_weights',dict(setting=f'w{w}-p{p}',**row))
        else:
            previous=self.models.get((kind,param))
            if previous is None or d==phase_start or d-previous[0]>=7:previous=self.fit(kind,param,d)
            fitday,fitid,models=previous
            assert fitday<=d
            forecasts=[]
            for var in ['load','pv']:
                x=np.vstack([self.data.features(d,h,var)[0] for h in [0,1]])
                model,scaler=models[var];pred=model.predict(scaler.transform(x) if scaler is not None else x)
                pred=np.maximum(pred,0.)
                if var=='pv':
                    support=np.any(self.data.a['pv'][max(0,d-28):d]>0,axis=0)
                    pred*=np.tile(support,2)
                forecasts.append(pred)
            l,v=forecasts
        assert np.isfinite(l).all() and np.isfinite(v).all() and min(l.min(),v.min())>=0
        self.event('forecast_registry',{'kind':kind,'parameter':str(param),'decision_date':daystr(d),
```

```
            'fit_id':fitid,'forecast_horizon_days':2,'latest_allowed_observation_end':daystr(d)+'
            ↪ 00:00',
            'prediction_sha256':hashlib.sha256(np.r_[l,v].tobytes()).hexdigest()})
        self.cache[key]=(l,v);return l,v

class Planner:
    def __init__(self,price,event,guard,hours=48,terminal=True):
        self.p=np.tile(price,hours//24);self.value=.9*price[0] if terminal else
        ↪ 0.;self.calls=0;self.event=event;self.guard=guard
        n=hours*6;self.n=n;eq=lil_matrix((n,4*n));ub=lil_matrix((n,4*n))
        for k in range(n):
            eq[k,k]=-.9;eq[k,n+k]=-.9;eq[k,2*n+k]=1/.9;eq[k,3*n+k]=1
            if k:eq[k,3*n+k-1]=-1
            ub[k,k]=ub[k,n+k]=1
        self.eq=eq.tocsr();self.ub=ub.tocsr();self.bub=np.full(n,5000/6)
        self.c=np.r_[self.p,np.zeros(n),-self.p,np.zeros(n)];self.c[-1]=-self.value

    def solve(self,L,V,state,margin,meta):
        self.guard();self.calls+=1
        if self.calls>8500:raise RuntimeError('LP hard budget exceeded')
        n=self.n;L=L[:n];V=V[:n];margin=margin[:n];net=np.maximum((L-V)/6+margin,0);excess=np.maximu
        ↪ m((V-L)/6-margin,0)
        lo=np.r_[np.zeros(3*n),np.full(n,1200.)]
        hi=np.r_[np.full(n,5000/6),np.minimum(excess,5000/6),np.minimum(net,5000/6),np.full(n,10800.
        ↪ )]
        beq=np.zeros(n);beq[0]=state;start=time.perf_counter()
        r=linprog(self.c,A_ub=self.ub,b_ub=self.bub,A_eq=self.eq,b_eq=beq,bounds=np.column_stack((lo
        ↪ ,hi)),
                  method='highs',options={'time_limit':10.,'primal_feasibility_tolerance':1e-8,'dual
                  ↪ _feasibility_tolerance':1e-8})
        elapsed=time.perf_counter()-start
        if not r.success:raise RuntimeError(r.message)
        x,y,z,s=np.split(r.x,4);q=net-z+x
        eqerr=np.max(np.abs(self.eq@r.x-beq));uberr=max(0.,np.max(self.ub@r.x-self.bub))
        berr=max(0.,np.max(lo-r.x),np.max(r.x-hi))
        dual=beq@r.eqlin.marginals+self.bub@r.ineqlin.marginals+lo@r.lower.marginals+hi@r.upper.marg
        ↪ inals
        gap=abs(r.fun-dual);sim=int(np.sum((x+y>1e-6)&(z>1e-6)))
        assert max(eqerr,uberr,berr)<1e-6 and gap<1e-4 and sim==0 and q.min()>-1e-6
        q=np.maximum(q,0)
        self.event('solver_checks',dict(**meta,status=int(r.status),seconds=elapsed,eq_residual=floa
        ↪ t(eqerr),
               inequality_residual=float(uberr),bound_residual=float(berr),primal_dual_gap=float(gap
               ↪ ),
               simultaneous_slots=sim,objective=float(r.fun+self.p@net),initial_storage=float(state),
               terminal_storage=float(s[-1]),terminal_value=float(self.value)))
        return q,x,y,z,s

def execute(L,V,q,initial,price):
    rows=[];state=float(initial)
    for k in range(144):
        load=float(L[k]/6);pv=float(V[k]/6);order=float(q[k]);m=min(load,pv);need=load-m;surplus=pv-m
        s0=state
        if need>order:
            z=min(need-order,5000/6,max(0.,.9*(state-1200)))
            e=max(0.,need-order-z);y=x=0.;received=order
        else:
```

```
            c=min(surplus+order-need,5000/6,max(0.,(10800-state)/.9))
            y=min(surplus,c);x=c-y;received=need+x;z=e=0.
        state+=.9*(x+y)-z/.9
        unused_order=max(0.,order-received);unused_pv=max(0.,surplus-y)
        bal=m+(received-x)+z+e-load
        assert abs(bal)<1e-6 and 1200-1e-6<=state<=10800+1e-6
        rows.append({'slot':k,'load_kw':float(L[k]),'pv_kw':float(V[k]),'price':float(price[k]),
                     'order_kwh':order,'received_kwh':received,'grid_to_load_kwh':received-x,
                     'direct_pv_kwh':m,'grid_charge_kwh':x,'pv_charge_kwh':y,'discharge_kwh':z,
                     'emergency_kwh':e,'unused_order_kwh':unused_order,'unused_pv_kwh':unused_pv,
                     'storage_start_kwh':s0,'storage_end_kwh':state,
                     'ordinary_cost':float(price[k]*order),'emergency_cost':float(5*price[k]*e),
                     'balance_residual':bal})
    return rows,state

def metrics(rows,Lhat,Vhat):
    df=pd.DataFrame(rows);r={n:float(df[n].sum()) for n in
    ↪  ['order_kwh','received_kwh','grid_charge_kwh',
            'pv_charge_kwh','discharge_kwh','emergency_kwh','unused_order_kwh','unused_pv_kwh','ord
            ↪  inary_cost','emergency_cost']}
    errorL=df.load_kw.to_numpy()-Lhat[:144];errorV=df.pv_kw.to_numpy()-Vhat[:144];pos=df.pv_kw.to_nu
    ↪  mpy()>0
    event=df.emergency_kwh.to_numpy()>1e-6
    longest=cur=0
    for yes in event:cur=cur+1 if yes else 0;longest=max(longest,cur)
    r.update(total_cost=r['ordinary_cost']+r['emergency_cost'],storage_start_kwh=rows[0]['storage_st
    ↪  art_kwh'],
            storage_end_kwh=rows[-1]['storage_end_kwh'],emergency_slots=int(event.sum()),
            longest_emergency_minutes=int(longest*10),load_abs_error_sum=float(abs(errorL).sum()),
            pv_abs_error_sum=float(abs(errorV).sum()),pv_positive_abs_error_sum=float(abs(errorV[po
            ↪  s]).sum()),
            pv_positive_slots=int(pos.sum()),net_abs_error_sum=float(abs(errorL-errorV).sum()),
            load_squared_error_sum=float((errorL**2).sum()),pv_squared_error_sum=float((errorV**2).
            ↪  sum()))
    return r

class Risk:
    def __init__(self,data,base=(4,7)):self.data=data;self.base=base;self.errors={}
    def error(self,d):
        if d not in self.errors:
            l,v,_=self.data.history(d,*self.base);a,b=self.data.truth(d);self.errors[d]=((a-b)-(l[:1
            ↪  44]-v[:144]))/6
        return self.errors[d]
    def margin(self,d,q=.7,days=28,group_hours=4,decay=None):
        ix=list(range(max(1,d-days),d));pool=np.array([self.error(i) for i in
        ↪  ix]);m=np.zeros(144);records=[]
        width=group_hours*6
        for j in range(0,144,width):
            vals=(pool[:,j:j+width] if len(ix)>=14 else pool).ravel()
            if len(ix)<7:raw=0.
            elif decay is None:raw=float(np.quantile(vals,q))
            else:
                w=np.repeat(decay**np.arange(len(ix)-1,-1,-1),width if len(ix)>=14 else 144)
                order=np.argsort(vals,kind='stable');cdf=np.cumsum(w[order]);raw=float(vals[order][n
                ↪  p.searchsorted(cdf,q*cdf[-1],side='left')])
            m[j:j+width]=max(0.,raw);records.append({'slot_start':j,'raw_quantile':raw,'value':max(0
            ↪  .,raw),'count':len(vals),'first':ix[0] if ix else None,'last':ix[-1] if ix else
            ↪  None})
```

```
        return np.tile(m,2),records
```

q3.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
import q3_base as base
class Planner(base.Planner):
    def solve(self,L,V,initial,margin,slot,q0=None,fixed=None,terminal=True,floor_ratio=0.):
        n=len(L);f=0 if q0 is None else min(144-slot,n);eq,ub=self.matrices(n,f)
        p=self.price[(slot+np.arange(n))%144];net=np.maximum((L-V)/6+margin,0);sur=np.maximum(-((L-V)/6+margin),0)
        c=np.zeros(8*n);c[4*n:5*n]=5*p;c[6*n:7*n]=p;c[7*n:7*n+f]=.5*p[:f]
        if terminal:c[4*n-1]=-.9*self.price[0]
        lo=np.zeros(8*n);hi=np.full(8*n,np.inf)
        hi[:n]=CAP;hi[n:2*n]=np.minimum(sur,CAP);hi[2*n:3*n]=CAP;lo[3*n:4*n]=1200;hi[3*n:4*n]=10800
        hi[7*n+f:]=0
        if f and floor_ratio>0:lo[6*n:6*n+f]=floor_ratio*np.asarray(q0[:f])
        if fixed is not None:lo[6*n:7*n]=fixed;hi[6*n:7*n]=fixed
        beq=np.r_[np.zeros(n),net];beq[0]=initial
        bu=np.r_[np.full(n,CAP),net,([] if f==0 else q0[:f]),([] if f==0 else -q0[:f])]
        self.budget.calls+=1;self.budget.check();start=time.perf_counter()
        ans=linprog(c,A_ub=ub,A_eq=eq,b_ub=bu,b_eq=beq,bounds=np.c_[lo,hi],method='highs',
            options={'time_limit':10.,'primal_feasibility_tolerance':1e-8,'dual_feasibility_tolerance':1e-8})
        if not ans.success:raise RuntimeError(ans.message)
        err=max(float(np.max(abs(eq@ans.x-beq))),float(np.max(ub@ans.x-bu)),float(np.max(lo-ans.x)),float(np.max(ans.x-hi)))
```

```
        finite=np.isfinite(hi)
        dual=beq@ans.eqlin.marginals+bu@ans.ineqlin.marginals+lo@ans.lower.marginals+hi[finite]@ans.upper.marginals[finite]
        gap=abs(ans.fun-dual);z=ans.x.reshape(8,n)
        simultaneous=int(np.sum((z[0]+z[1]>1e-6)&(z[2]>1e-6)))
        assert err<1e-6 and gap<1e-4 and simultaneous==0
        return z,dict(n=n,f=f,initial=float(initial),objective=float(ans.fun),residual=err,dual_gap=float(gap),
            simultaneous=simultaneous,seconds=time.perf_counter()-start,status=int(ans.status),floor_ratio=float(floor_ratio))

def exact_quantile(values,ages,q,beta):
    from fractions import Fraction
    qf=Fraction(str(q));bf=Fraction(str(beta));age_max=int(np.max(ages))
    weights=[bf.numerator**int(a)*bf.denominator**(age_max-int(a)) for a in ages]
    total=sum(weights);acc=0
    for i,w in enumerate(weights):
        acc+=w
        if acc*qf.denominator>=qf.numerator*total:return max(0.,float(values[i]))
    return max(0.,float(values[-1]))
def make_load(kind,gain):
    residual=src['actual'][:365*144,0].reshape(365,144)-fc['L'][:,:144]
    load=np.full((1460,288),np.nan);theta=np.zeros((365,4,8));obs=np.zeros((365,4));count=np.zeros((365,4,8),int)
    for d in range(365):
        for a,slot in enumerate((0,36,72,108)):
            b=d*4+a;n=288-slot;load[b,:n]=fc['L'][d,slot:]
            if not slot:continue
            start=0 if 'PREFIX' in kind else slot-18
            current=residual[d,start:slot]
            observed=float(np.nanmean(current)) if np.isfinite(current).any() else 0.;obs[d,a]=observed
            past=np.arange(max(0,d-28),d)
            X=np.array([np.nanmean(residual[j,start:slot]) for j in past])
            for g in range(slot//18,8):
                Y=np.array([np.mean(residual[j,g*18:(g+1)*18]) for j in past])
                valid=np.isfinite(X)&np.isfinite(Y);x=X[valid];y=Y[valid];count[d,a,g]=len(x)
                if kind=='PERSIST_PREFIX':coef=1.
                else:coef=float(np.clip((x@y)/(x@x),0,1)) if len(x)>=7 and x@x>1e-12 else 0.
                theta[d,a,g]=coef
                left=g*18-slot;right=left+18
                load[b,left:right]=np.maximum(0,load[b,left:right]+gain*coef*observed)
    return load,theta,obs,count

def errors_for(load,pv):
    arrays={k:[] for k in ('release','target','error','kind')}
    for b,r in enumerate(src['release']):
        n=288-r%144;t=r+np.arange(n);ok=(t>0)&(t<365*144);truth=src['actual'][t[ok]]
        arrays['release'].append(np.full(ok.sum(),r));arrays['target'].append(t[ok]);arrays['kind'].append((np.arange(n)[ok]>=144).astype(int))
        arrays['error'].append(((truth[:,0]-truth[:,1])-(load[b,:n][ok]-pv[b,:n][ok]))/6)
    return {k:np.concatenate(v) for k,v in arrays.items()}
def midnight(family):
 if family=='W4':return orig['L'].copy()
 weeks=1 if family=='W1' else 2 if family=='W2' else 4
 decay=.5 if family=='EW4' else 1.
 L=np.empty((365,288))
 for d in range(365):
```

```
  for h in (0,1):
   ds=[d+h-7*j for j in range(1,weeks+1) if 0<=d+h-7*j<d]
   if not ds:ds=list(range(max(0,d-7),d));weights=np.ones(len(ds))
   else:weights=decay**np.arange(len(ds))
   if not ds:L[d,h*144:(h+1)*144]=src['reference'][:,0];continue
   z=src['actual'][np.asarray(ds)[:,None]*144+np.arange(144),0];valid=np.isfinite(z)
   den=(weights[:,None]*valid).sum(0);num=(weights[:,None]*np.nan_to_num(z)).sum(0)
   L[d,h*144:(h+1)*144]=np.divide(num,den,out=src['reference'][:,0].copy(),where=den>0)
 return L

def group_index(err):
 global group_cache
 if group_cache is not None:return group_cache
 rr,tt,kind=(err[k] for k in ('release','target','kind'));bank=[]
 for b,r in enumerate(src['release']):
  n=288-r%144;t=r+np.arange(n);pool=np.flatnonzero((rr>=r-4032)&(rr<r)&(tt+1<=r)&(rr%144==r%144))
  for typ in (0,1):
   same=pool[kind[pool]==typ]
   for g in range(8):
    dest=np.flatnonzero((t%144//18==g)&((np.arange(n)>=144)==bool(typ)))
    if not len(dest):continue
    ids=same[tt[same]%144//18==g];fallback=0
    if len(ids)<24:ids=same;fallback=1
    bank.append((b,typ,g,dest,ids,(r-1)//144-tt[ids]//144,fallback))
 group_cache=bank;return bank

def quantile(values,ages,q):
 from fractions import Fraction
 order=np.argsort(values,kind='stable');values=values[order];ages=ages[order];m=int(ages.max())
 iw=[7**a*10**(m-a) for a in range(m+1)];hist=np.bincount(ages,minlength=m+1)
 total=sum(int(h)*w for h,w in zip(hist,iw));f=Fraction(str(q))
 cdf=np.cumsum(.7**ages);k=min(len(values)-1,int(np.searchsorted(cdf/cdf[-1],q)))
 histpre=np.bincount(ages[:k+1],minlength=m+1);acc=sum(int(h)*w for h,w in zip(histpre,iw))
 while k>0 and f.denominator*(acc-iw[int(ages[k])])>=f.numerator*total:acc-=iw[int(ages[k])];k-=1
 while f.denominator*acc<f.numerator*total:k+=1;acc+=iw[int(ages[k])]
 return max(0.,float(values[k]))
```

q3_base.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
```

```
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
def history_and_features(actual,reference,F,release,targets):
    # Profile j only uses observations of that clock slot whose intervals ended by r.
    hist=np.empty((1460,144));last=np.empty(1460)
    for b,r in enumerate(release):
        last[b]=actual[r-1,1] if r>1 else reference[0,1]
        for k in range(144):
            q=r-1-((r-1-k)%144);ii=q-144*np.arange(7);ii=ii[ii>=1]
            hist[b,k]=np.mean(actual[ii,1]) if len(ii) else reference[k,1]
    L=np.empty((365,288))
    for d in range(365):
        for h in [0,1]:
            ds=[d+h-7*j for j in range(1,5) if 0<=d+h-7*j<d]
            if not ds:ds=list(range(max(0,d-7),d))
            if not ds:L[d,h*144:(h+1)*144]=reference[:,0];continue
            z=actual[np.array(ds)[:,None]*144+np.arange(144)[None,:],0]
            cnt=np.isfinite(z).sum(axis=0)
            L[d,h*144:(h+1)*144]=np.divide(np.nansum(z,axis=0),cnt,out=reference[:,0].copy(),where=cnt>0)
    H=np.take_along_axis(hist,targets%144,axis=1)
    hr=targets%144/6;year=targets//144+1
    X=np.stack([F,H,np.repeat(last[:,None],24,axis=1),np.repeat((release%144/6)[:,None],24,axis=1),
                np.tile(np.arange(1,25),(1460,1)),np.sin(2*np.pi*hr/24),np.cos(2*np.pi*hr/24),
                np.sin(2*np.pi*year/365),np.cos(2*np.pi*year/365)],axis=-1).reshape(-1,9)
    return hist,last,L,H,X

def build_fusion(actual,F,H,release,targets):
    rr=np.repeat(release,24);tt=targets.ravel();ff=F.ravel();hh=H.ravel()
    truth=np.full(len(tt),np.nan);valid=tt<N;truth[valid]=actual[tt[valid],1]
    w=np.ones((1460,4));counts=np.zeros((1460,4),int);fallback=np.zeros((1460,4),int);cut=np.full((1460,4),-1,int)
    leadgroup=np.tile(np.arange(24)//6,1460)
    for b,r in enumerate(release):
        pool=(rr>=r-28*144)&(rr<r)&(tt+1<=r)&(rr%144==r%144)&np.isfinite(truth)
        for g in range(4):
            use=pool&(leadgroup==g)
            if use.sum()<24:use=pool;fallback[b,g]=1
            counts[b,g]=use.sum()
            if use.any():cut[b,g]=int(np.max(tt[use]+1))
            a=ff[use]-hh[use];den=a@a
            if use.sum()<24 or den<1e-12:fallback[b,g]=2;continue
            w[b,g]=np.clip(a@(truth[use]-hh[use])/den,0,1)
    fused=np.repeat(w,6,axis=1)*F+(1-np.repeat(w,6,axis=1))*H
    return fused,w,counts,fallback,cut

def to_intervals(hourly,hist,last,release,zero_version=False):
```

```
    out=np.full((1460,288),np.nan)
    for b,r in enumerate(release):
        n=288-r%144;targets=r+np.arange(n);v=hist[b,targets%144].copy()
        if zero_version:
            midnight=(r//144)*144;ix=np.flatnonzero(midnight+np.arange(1,25)*6>r)
            xp=np.r_[r,midnight+(ix+1)*6];fp=np.r_[last[b],hourly[(r//144)*4,ix]]
            inside=targets<midnight+144
            v[inside]=np.interp(targets[inside],xp,fp)
        else:v[:144]=np.interp(np.arange(144),np.arange(0,145,6),np.r_[last[b],hourly[b]])
        out[b,:n]=v
    return out

class Planner:
    """Variables x,y,z,S,e,r,q,w; r is paid but unreceived normal energy."""
    def __init__(self,price,budget):self.price=price;self.budget=budget;self.cache={}
    def matrices(self,n,f):
        key=(n,f)
        if key in self.cache:return self.cache[key]
        eq=lil_matrix((2*n,8*n));ub=lil_matrix((2*n+2*f,8*n))
        for i in range(n):
            eq[i,i]=-.9;eq[i,n+i]=-.9;eq[i,2*n+i]=1/.9;eq[i,3*n+i]=1
            if i:eq[i,3*n+i-1]=-1
            eq[n+i,6*n+i]=1;eq[n+i,i]=-1;eq[n+i,2*n+i]=1;eq[n+i,4*n+i]=1;eq[n+i,5*n+i]=-1
            ub[i,i]=ub[i,n+i]=1;ub[n+i,2*n+i]=ub[n+i,4*n+i]=1
        for i in range(f):
            ub[2*n+i,6*n+i]=1;ub[2*n+i,7*n+i]=-1
            ub[2*n+f+i,6*n+i]=-1;ub[2*n+f+i,7*n+i]=-1
        self.cache[key]=eq.tocsr(),ub.tocsr();return self.cache[key]
    def solve(self,L,V,initial,margin,slot,q0=None,fixed=None,terminal=True):
        n=len(L);f=0 if q0 is None else min(144-slot,n);eq,ub=self.matrices(n,f)
        p=self.price[(slot+np.arange(n))%144];net=np.maximum((L-V)/6+margin,0);ex=np.maximum((V-L)/6
        ↪  -margin,0)
        c=np.zeros(8*n);c[4*n:5*n]=5*p;c[6*n:7*n]=p;c[7*n:7*n+f]=.5*p[:f]
        if terminal:c[4*n-1]=-.9*self.price[0]
        lo=np.zeros(8*n);hi=np.full(8*n,np.inf)
        hi[:n]=CAP;hi[n:2*n]=np.minimum(ex,CAP);hi[2*n:3*n]=CAP;lo[3*n:4*n]=1200;hi[3*n:4*n]=10800
        hi[7*n+f:]=0
        if fixed is not None:lo[6*n:7*n]=fixed;hi[6*n:7*n]=fixed
        beq=np.r_[np.zeros(n),net];beq[0]=initial
        bu=np.r_[np.full(n,CAP),net,([] if f==0 else q0[:f]),([] if f==0 else -q0[:f])]
        self.budget.calls+=1;self.budget.check();start=time.perf_counter()
        ans=linprog(c,A_ub=ub,b_ub=bu,A_eq=eq,b_eq=beq,bounds=np.c_[lo,hi],method='highs',
              options={'time_limit':10.,'primal_feasibility_tolerance':1e-8,'dual_feasibility_tolera
              ↪  nce':1e-8})
        if not ans.success:raise RuntimeError(ans.message)
        err=max(float(np.max(abs(eq@ans.x-beq))),float(np.max(ub@ans.x-bu)),float(np.max(lo-ans.x)),
        ↪  float(np.max(ans.x-hi)))
        finite=np.isfinite(hi)
        dual=beq@ans.eqlin.marginals+bu@ans.ineqlin.marginals+lo@ans.lower.marginals+hi[finite]@ans.
        ↪  upper.marginals[finite]
        gap=abs(ans.fun-dual);z=ans.x.reshape(8,n)
        simultaneous=int(np.sum((z[0]+z[1]>1e-6)&(z[2]>1e-6)))
        assert err<1e-6 and gap<1e-4,(err,gap)
        # Fixed-order feasibility diagnostics may be degenerate; actual execution is independently
        ↪  causal.
        if fixed is None:assert simultaneous==0, 'planned simultaneous charge/discharge'
        check=dict(n=n,f=f,initial=float(initial),objective=float(ans.fun),residual=err,dual_gap=flo
        ↪  at(gap),
```

```
                    simultaneous=simultaneous,seconds=time.perf_counter()-start,status=int(ans.status
                    ↪  ))
        return z,check

def execute_block(L,V,q,initial):
    # Columns retain physical flows separately from paid order and settlement.
    out=np.empty((len(q),12));s=float(initial)
    for k in range(len(q)):
        l=float(L[k]/6);v=float(V[k]/6);order=max(0.,float(q[k]));direct=min(l,v);need=l-direct;surp
        ↪  lus=v-direct;s0=s
        if need>order:
            z=min(need-order,CAP,max(0.,.9*(s-1200)));e=max(0.,need-order-z);x=y=0.;received=order
        else:
            ch=min(surplus+order-need,CAP,max(0.,(10800-s)/.9));y=min(surplus,ch);x=ch-y;received=ne
            ↪  ed+x;z=e=0.
        s+=.9*(x+y)-z/.9
        out[k]=[s0,s,received,received-x,direct,x,y,z,e,max(0.,order-received),max(0.,surplus-y),l]
        assert 1200-1e-6<=s<=10800+1e-6 and abs(received-x+direct+z+e-l)<1e-6
    return out,s
```

q4.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
EXEC_COLUMNS=['storage_start','storage_end','received','grid_load','pv_load','grid_charge','pv_charg
↪  e','discharge','emergency','unused_order','unused_pv','load_energy']
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False
 ↪  ,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
class Q2Risk:
```

```
    def __init__(self, data):
        self.data = data
        self.errors = {}

    def error(self, day):
        if day not in self.errors:
            load_hat, pv_hat, _ = self.data.history(day, 4, 7)
            load, pv = self.data.truth(day)
            self.errors[day] = ((load - pv) - (load_hat[:SLOTS] - pv_hat[:SLOTS])) / 6
        return self.errors[day]

    def margin(self, day, q=0.7, history_days=28, group_hours=4, decay=0.7):
        indices = list(range(max(1, day - history_days), day))
        pool = np.array([self.error(index) for index in indices])
        result = np.zeros(SLOTS)
        width = group_hours * 6
        records = []
        for start in range(0, SLOTS, width):
            values = (pool[:, start:start + width] if len(indices) >= 14 else pool).ravel()
            if len(indices) < 7:
                raw = 0.0
                count = int(values.size)
            else:
                weights = np.repeat(
                    decay ** np.arange(len(indices) - 1, -1, -1),
                    width if len(indices) >= 14 else SLOTS,
                )
                order = np.argsort(values, kind="stable")
                cdf = np.cumsum(weights[order])
                raw = float(values[order][np.searchsorted(cdf, q * cdf[-1], side="left")])
                count = int(values.size)
            result[start:start + width] = max(0.0, raw)
            records.append({"slot_start": start, "value": max(0.0, raw), "count": count})
        return np.tile(result, 2), records

class PriceFeatures:
    FEATURE_NAMES = [
        "attachment1_base", "matching_weekday_median4", "recent7_median", "lag7_price",
        "predicted_net_load_1000kw", "current_day_level", "current_level_decay6h",
        "last_residual_decay1h", "lead_hours", "season_sin", "season_cos",
        *[f"weekday_{i}" for i in range(7)], "release_fraction",
    ]

    def __init__(self, prices, base, supply, branch):
        self.prices = prices
        self.base = base
        self.supply = supply
        self.branch = branch
        self.releases = [0] if branch == "Q4-2" else [0, 36, 72, 108]
        self._history_cache = {}

    def supply_forecast(self, day, slot):
        if self.branch == "Q4-2":
            target = day * SLOTS + np.arange(288)
            net = self.supply["load"][day] - self.supply["pv"][day]
        else:
            batch = day * 4 + slot // 36
            n = 288 - slot
```

```
            target = day * SLOTS + slot + np.arange(n)
            net = self.supply["load"][day, slot:slot + n] - self.supply["pv"][batch, :n]
        return target.astype(int), np.asarray(net, dtype=float)

    def history_curve(self, decision_day, target_day):
        key = (int(decision_day), int(target_day))
        if key in self._history_cache:
            return self._history_cache[key]
        matching = [target_day - 7 * lag for lag in range(1, 5) if 0 <= target_day - 7 * lag <
        ↪  decision_day]
        if matching:
            curve = np.median(self.prices[matching], axis=0)
            source = "matching_weekday"
        else:
            recent = list(range(max(0, decision_day - 7), decision_day))
            curve = np.median(self.prices[recent], axis=0) if recent else self.base.copy()
            source = "recent_or_attachment1"
        self._history_cache[key] = (curve, matching, source)
        return self._history_cache[key]

    def simple(self, day, slot, targets=None):
        if targets is None:
            targets, _ = self.supply_forecast(day, slot)
        slots = targets % SLOTS
        days = targets // SLOTS
        out = np.empty(len(targets))
        for target_day in np.unique(days):
            mask = days == target_day
            curve, _, _ = self.history_curve(day, int(target_day))
            out[mask] = curve[slots[mask]]
        if slot > 0:
            current_curve, _, _ = self.history_curve(day, day)
            level = float(np.median(self.prices[day, :slot] - current_curve[:slot]))
            out[days == day] += level
        return np.maximum(out, 0.001)

    def matrix(self, day, slot, targets, predicted_net):
        target_slots = targets % SLOTS
        target_days = targets // SLOTS
        same = np.empty(len(targets))
        recent = np.empty(len(targets))
        lag7 = np.empty(len(targets))
        for target_day in np.unique(target_days):
            mask = target_days == target_day
            slots = target_slots[mask]
            curve, _, _ = self.history_curve(day, int(target_day))
            same[mask] = curve[slots]
            recent_days = list(range(max(0, day - 7), day))
            recent[mask] = np.median(self.prices[recent_days][:, slots], axis=0) if recent_days else
            ↪  self.base[slots]
            lag_day = int(target_day) - 7
            lag7[mask] = self.prices[lag_day, slots] if 0 <= lag_day < day else curve[slots]

        current_level = 0.0
        last_residual = 0.0
        if slot > 0:
            current_curve, _, _ = self.history_curve(day, day)
            residual = self.prices[day, :slot] - current_curve[:slot]
```

```
            current_level = float(np.median(residual))
            last_residual = float(residual[-1])
        lead_hours = (targets - (day * SLOTS + slot)) / 6
        same_day = (target_days == day).astype(float)
        current = current_level * same_day
        current_decay = current_level * np.exp(-lead_hours / 6) * same_day
        last_decay = last_residual * np.exp(-lead_hours) * same_day
        dates = DATE + pd.to_timedelta(target_days, unit="D")
        day_of_year = np.asarray(dates.dayofyear, dtype=float)
        weekday = np.asarray(dates.dayofweek, dtype=int)
        columns = [
            self.base[target_slots], same, recent, lag7, predicted_net / 1000,
            current, current_decay, last_decay, lead_hours,
            np.sin(2 * np.pi * day_of_year / 365), np.cos(2 * np.pi * day_of_year / 365),
        ]
        columns.extend((weekday == index).astype(float) for index in range(7))
        columns.append(np.full(len(targets), slot / 108 if slot else 0.0))
        matrix = np.column_stack(columns)
        assert matrix.shape[1] == len(self.FEATURE_NAMES) and np.isfinite(matrix).all()
        return matrix

class PriceModels:
    def __init__(self, features, branch):
        self.features = features
        self.branch = branch
        self.datasets = {}
        self.models = {}
        self.model_dir = O / "price_models" / branch.replace("-", "_")
        self.model_dir.mkdir(parents=True, exist_ok=True)

    def training(self, cutoff_day):
        if cutoff_day in self.datasets:
            return self.datasets[cutoff_day]
        rows, targets = [], []
        for origin_day in range(max(1, cutoff_day - 56), cutoff_day):
            for release in self.features.releases:
                absolute, net = self.features.supply_forecast(origin_day, release)
                valid = (absolute < cutoff_day * SLOTS) & (absolute < N_DAYS * SLOTS)
                if not valid.any():
                    continue
                rows.append(self.features.matrix(origin_day, release, absolute[valid], net[valid]))
                targets.append(self.features.prices.reshape(-1)[absolute[valid]])
        X = np.vstack(rows)
        y = np.concatenate(targets)
        assert len(y) >= 500 and np.isfinite(X).all() and np.isfinite(y).all()
        self.datasets[cutoff_day] = (X, y)
        return X, y

    def fit(self, candidate, cutoff_day):
        key = (candidate, cutoff_day)
        if key in self.models:
            return self.models[key]
        X, y = self.training(cutoff_day)
        start = time.perf_counter()
        budget.price_fits += 1
        budget.check()
        if candidate == "P2":
            scaler = StandardScaler()
```

```
            scaled = scaler.fit_transform(X)
            model = Ridge(alpha=10.0)
            with threadpool_limits(limits=1):
                model.fit(scaled, y)
            original_coef = model.coef_ / scaler.scale_
            record = {
                "coefficients_original": dict(zip(self.features.FEATURE_NAMES,
                ↪  original_coef.astype(float))),
                "intercept_original": float(model.intercept_ - original_coef @ scaler.mean_),
                "alpha": 10.0,
            }
        else:
            scaler = None
            model = HistGradientBoostingRegressor(
                max_leaf_nodes=15, max_iter=100, learning_rate=0.05,
                min_samples_leaf=50, l2_regularization=1.0,
                early_stopping=False, random_state=2026,
            )
            with threadpool_limits(limits=1):
                model.fit(X, y)
            record = {
                "max_leaf_nodes": 15, "iterations": int(model.n_iter_),
                "learning_rate": 0.05, "min_samples_leaf": 50, "random_state": 2026,
            }
        elapsed = time.perf_counter() - start
        path = self.model_dir / f"{candidate}-cutoff-{cutoff_day:03d}.joblib"
        joblib.dump({"model": model, "scaler": scaler, "features": self.features.FEATURE_NAMES},
        ↪  path, compress=3)
        record.update({
            "branch": self.branch, "candidate": candidate, "cutoff_day": int(cutoff_day),
            "cutoff_datetime": daystr(cutoff_day) + " 00:00", "training_rows": int(len(y)),
            "training_origin_start": daystr(max(1, cutoff_day - 56)),
            "latest_target_end": daystr(cutoff_day) + " 00:00",
            "seconds": elapsed, "model_path": str(path.relative_to(R)), "model_sha256": sha(path),
        })
        logs.event("price_fit_log", record)
        self.models[key] = (model, scaler)
        budget.save()
        return self.models[key]

    def predict(self, candidate, day, slot):
        targets, net = self.features.supply_forecast(day, slot)
        if candidate == "P1" or day < 7:
            result = self.features.simple(day, slot, targets)
            fit_cutoff = None
            fallback = candidate != "P1"
        else:
            cutoff = 7 + ((day - 7) // 7) * 7
            model, scaler = self.fit(candidate, cutoff)
            X = self.features.matrix(day, slot, targets, net)
            with threadpool_limits(limits=1):
                result = model.predict(scaler.transform(X) if scaler is not None else X)
            result = np.maximum(result, 0.001)
            fit_cutoff = cutoff
            fallback = False
        logs.event("price_forecast_log", {
            "branch": self.branch, "candidate": candidate, "decision_day": int(day),
            "decision_date": daystr(day), "release_slot": int(slot),
```

```
            "known_price_intervals_today": int(slot), "fit_cutoff_day": fit_cutoff,
            "early_history_fallback": fallback, "forecast_points": int(len(result)),
            "minimum": float(np.min(result)), "maximum": float(np.max(result)),
            "sha256": hashlib.sha256(np.asarray(result, dtype=np.float64).tobytes()).hexdigest(),
        })
        return result

class Q42Planner:
    def __init__(self, terminal_value):
        n = 288
        self.n = n
        self.terminal_value = float(terminal_value)
        eq = lil_matrix((n, 4 * n))
        ub = lil_matrix((n, 4 * n))
        for k in range(n):
            eq[k, k] = -0.9
            eq[k, n + k] = -0.9
            eq[k, 2 * n + k] = 1 / 0.9
            eq[k, 3 * n + k] = 1
            if k:
                eq[k, 3 * n + k - 1] = -1
            ub[k, k] = ub[k, n + k] = 1
        self.eq = eq.tocsr()
        self.ub = ub.tocsr()
        self.bub = np.full(n, CAP)

    def solve(self, load, pv, state, margin, price, meta):
        budget.lp_calls += 1
        budget.check()
        n = self.n
        net = np.maximum((load[:n] - pv[:n]) / 6 + margin[:n], 0)
        excess = np.maximum((pv[:n] - load[:n]) / 6 - margin[:n], 0)
        lower = np.r_[np.zeros(3 * n), np.full(n, 1200.0)]
        upper = np.r_[np.full(n, CAP), np.minimum(excess, CAP), np.minimum(net, CAP), np.full(n,
        ↪  10800.0)]
        beq = np.zeros(n)
        beq[0] = state
        objective = np.r_[price[:n], np.zeros(n), -price[:n], np.zeros(n)]
        objective[-1] = -self.terminal_value
        start = time.perf_counter()
        result = linprog(
            objective, A_ub=self.ub, b_ub=self.bub, A_eq=self.eq, b_eq=beq,
            bounds=np.column_stack((lower, upper)), method="highs",
            options={"time_limit": 10.0, "primal_feasibility_tolerance": 1e-8,
            ↪  "dual_feasibility_tolerance": 1e-8},
        )
        if not result.success:
            raise RuntimeError(result.message)
        x, y, z, storage = np.split(result.x, 4)
        order = np.maximum(net - z + x, 0)
        residual = max(
            float(np.max(np.abs(self.eq @ result.x - beq))),
            float(np.max(self.ub @ result.x - self.bub)),
            float(np.max(lower - result.x)), float(np.max(result.x - upper)),
        )
        finite = np.isfinite(upper)
        dual = (beq @ result.eqlin.marginals + self.bub @ result.ineqlin.marginals
                + lower @ result.lower.marginals + upper[finite] @ result.upper.marginals[finite])
```

```
        gap = abs(result.fun - dual)
        simultaneous = int(np.sum((x + y > 1e-6) & (z > 1e-6)))
        assert residual < 1e-6 and gap < 1e-4 and simultaneous == 0
        logs.event("solver_checks", dict(meta, residual=residual, dual_gap=float(gap),
                                         simultaneous=simultaneous, status=int(result.status),
                                         seconds=time.perf_counter() - start))
        return order

class Q43Planner:
    def __init__(self, terminal_value):
        self.terminal_value = float(terminal_value)
        self.cache = {}

    def matrices(self, n, fixed_count):
        key = (n, fixed_count)
        if key in self.cache:
            return self.cache[key]
        eq = lil_matrix((2 * n, 8 * n))
        ub = lil_matrix((2 * n + 2 * fixed_count, 8 * n))
        for i in range(n):
            eq[i, i] = -0.9
            eq[i, n + i] = -0.9
            eq[i, 2 * n + i] = 1 / 0.9
            eq[i, 3 * n + i] = 1
            if i:
                eq[i, 3 * n + i - 1] = -1
            eq[n + i, 6 * n + i] = 1
            eq[n + i, i] = -1
            eq[n + i, 2 * n + i] = 1
            eq[n + i, 4 * n + i] = 1
            eq[n + i, 5 * n + i] = -1
            ub[i, i] = ub[i, n + i] = 1
            ub[n + i, 2 * n + i] = ub[n + i, 4 * n + i] = 1
        for i in range(fixed_count):
            ub[2 * n + i, 6 * n + i] = 1
            ub[2 * n + i, 7 * n + i] = -1
            ub[2 * n + fixed_count + i, 6 * n + i] = -1
            ub[2 * n + fixed_count + i, 7 * n + i] = -1
        self.cache[key] = eq.tocsr(), ub.tocsr()
        return self.cache[key]

    def solve(self, load, pv, state, margin, slot, q0, price, meta):
        budget.lp_calls += 1
        budget.check()
        n = len(load)
        fixed_count = 0 if q0 is None else min(SLOTS - slot, n)
        eq, ub = self.matrices(n, fixed_count)
        net = np.maximum((load - pv) / 6 + margin, 0)
        surplus = np.maximum(-((load - pv) / 6 + margin), 0)
        objective = np.zeros(8 * n)
        objective[4 * n:5 * n] = 5 * price
        objective[6 * n:7 * n] = price
        objective[7 * n:7 * n + fixed_count] = 0.5 * price[:fixed_count]
        objective[4 * n - 1] = -self.terminal_value
        lower = np.zeros(8 * n)
        upper = np.full(8 * n, np.inf)
        upper[:n] = CAP
        upper[n:2 * n] = np.minimum(surplus, CAP)
```

```
        upper[2 * n:3 * n] = CAP
        lower[3 * n:4 * n] = 1200
        upper[3 * n:4 * n] = 10800
        upper[7 * n + fixed_count:] = 0
        beq = np.r_[np.zeros(n), net]
        beq[0] = state
        bub = np.r_[np.full(n, CAP), net,
                    ([] if fixed_count == 0 else q0[:fixed_count]),
                    ([] if fixed_count == 0 else -q0[:fixed_count])]
        start = time.perf_counter()
        result = linprog(
            objective, A_ub=ub, b_ub=bub, A_eq=eq, b_eq=beq,
            bounds=np.column_stack((lower, upper)), method="highs",
            options={"time_limit": 10.0, "primal_feasibility_tolerance": 1e-8,
            ↪  "dual_feasibility_tolerance": 1e-8},
        )
        if not result.success:
            raise RuntimeError(result.message)
        z = result.x.reshape(8, n)
        residual = max(
            float(np.max(np.abs(eq @ result.x - beq))), float(np.max(ub @ result.x - bub)),
            float(np.max(lower - result.x)), float(np.max(result.x - upper)),
        )
        finite = np.isfinite(upper)
        dual = (beq @ result.eqlin.marginals + bub @ result.ineqlin.marginals
                + lower @ result.lower.marginals + upper[finite] @ result.upper.marginals[finite])
        gap = abs(result.fun - dual)
        simultaneous = int(np.sum((z[0] + z[1] > 1e-6) & (z[2] > 1e-6)))
        assert residual < 1e-6 and gap < 1e-4 and simultaneous == 0
        logs.event("solver_checks", dict(meta, residual=residual, dual_gap=float(gap),
                                         simultaneous=simultaneous, status=int(result.status),
                                         seconds=time.perf_counter() - start))
        return np.maximum(z[6], 0)

def execute_block(load_kw, pv_kw, order, initial):
    output = np.empty((len(order), len(EXEC_COLUMNS)))
    state = float(initial)
    for k in range(len(order)):
        load = float(load_kw[k] / 6)
        pv = float(pv_kw[k] / 6)
        paid = max(0.0, float(order[k]))
        direct = min(load, pv)
        need = load - direct
        surplus = pv - direct
        start = state
        if need > paid:
            discharge = min(need - paid, CAP, max(0.0, 0.9 * (state - 1200)))
            emergency = max(0.0, need - paid - discharge)
            grid_charge = pv_charge = 0.0
            received = paid
        else:
            charge = min(surplus + paid - need, CAP, max(0.0, (10800 - state) / 0.9))
            pv_charge = min(surplus, charge)
            grid_charge = charge - pv_charge
            received = need + grid_charge
            discharge = emergency = 0.0
        state += 0.9 * (grid_charge + pv_charge) - discharge / 0.9
        output[k] = [
```

```
            start, state, received, received - grid_charge, direct, grid_charge, pv_charge,
            discharge, emergency, max(0.0, paid - received), max(0.0, surplus - pv_charge), load,
        ]
        assert 1200 - 1e-6 <= state <= 10800 + 1e-6
        assert abs(received - grid_charge + direct + discharge + emergency - load) < 1e-6
    return output, state
```

q4_tree.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False
 ↪  ,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
import q4
class ForecastEngine:
    """Rolling P3 direct-price or P2 residual-price model."""

    def __init__(self, name, features, model_kind, window=56, alpha=10.0,
                 feature_mode="full", max_leaf_nodes=15):
        self.name = name
        self.f = features
        self.model_kind = model_kind
        self.window = int(window)
        self.alpha = float(alpha)
        self.feature_mode = feature_mode
        self.max_leaf_nodes = int(max_leaf_nodes)
        self.cache = {}
        self.model_dir = O / "price_models" / name
        self.model_dir.mkdir(parents=True, exist_ok=True)
        self.index = {name: i for i, name in enumerate(self.f.FEATURE_NAMES)}
```

```
        if model_kind == "p3":
            if feature_mode == "full":
                self.feature_names = list(self.f.FEATURE_NAMES)
            elif feature_mode == "no_net":
                self.feature_names = [x for x in self.f.FEATURE_NAMES if x !=
                ↪  "predicted_net_load_1000kw"]
            elif feature_mode == "history_only":
                self.feature_names = list(self.f.FEATURE_NAMES[:4])
            else:
                raise ValueError(feature_mode)
        else:
            self.feature_names = [
                "matching_weekday_level", "recent_minus_matching", "lag7_minus_matching",
                "attachment1_minus_matching", "predicted_net_load_1000kw",
                "current_day_level", "current_level_decay6h", "last_residual_decay1h",
                "lead_hours", "season_sin", "season_cos",
                *[f"weekday_{i}" for i in range(7)], "release_fraction",
            ]
            if feature_mode == "residual_no_net":
                self.feature_names.remove("predicted_net_load_1000kw")

    def transform(self, full):
        if self.model_kind == "p3":
            return full[:, [self.index[x] for x in self.feature_names]], full[:,
            ↪  self.index["matching_weekday_median4"]]
        match = full[:, self.index["matching_weekday_median4"]]
        cols = {
            "matching_weekday_level": match,
            "recent_minus_matching": full[:, self.index["recent7_median"]] - match,
            "lag7_minus_matching": full[:, self.index["lag7_price"]] - match,
            "attachment1_minus_matching": full[:, self.index["attachment1_base"]] - match,
        }
        for name in (
            "predicted_net_load_1000kw", "current_day_level", "current_level_decay6h",
            "last_residual_decay1h", "lead_hours", "season_sin", "season_cos",
            *[f"weekday_{i}" for i in range(7)], "release_fraction",
        ):
            cols[name] = full[:, self.index[name]]
        return np.column_stack([cols[x] for x in self.feature_names]), match

    def training(self, cutoff):
        xs, ys = [], []
        for origin in range(max(1, cutoff - self.window), cutoff):
            for release in self.f.releases:
                targets, net = self.f.supply_forecast(origin, release)
                valid = (targets < cutoff * q4.SLOTS) & (targets < q4.N_DAYS * q4.SLOTS)
                if not valid.any():
                    continue
                full = self.f.matrix(origin, release, targets[valid], net[valid])
                x, match = self.transform(full)
                y = self.f.prices.reshape(-1)[targets[valid]]
                xs.append(x)
                ys.append(y if self.model_kind == "p3" else y - match)
        x = np.vstack(xs)
        y = np.concatenate(ys)
        assert len(y) >= 500 and np.isfinite(x).all() and np.isfinite(y).all()
        return x, y

```

```
    def fit(self, cutoff):
        if cutoff in self.cache:
            return self.cache[cutoff]
        x, y = self.training(cutoff)
        budget.price_fits += 1
        budget.check()
        started = time.perf_counter()
        if self.model_kind == "p3":
            scaler = None
            model = HistGradientBoostingRegressor(
                max_leaf_nodes=self.max_leaf_nodes, max_iter=100, learning_rate=0.05,
                min_samples_leaf=50, l2_regularization=1.0,
                early_stopping=False, random_state=2026,
            )
            with threadpool_limits(limits=1):
                model.fit(x, y)
            detail = {"max_leaf_nodes": self.max_leaf_nodes, "iterations": int(model.n_iter_),
                      "learning_rate": 0.05, "min_samples_leaf": 50}
        else:
            scaler = StandardScaler()
            z = scaler.fit_transform(x)
            model = Ridge(alpha=self.alpha)
            with threadpool_limits(limits=1):
                model.fit(z, y)
            original = model.coef_ / scaler.scale_
            detail = {
                "alpha": self.alpha,
                "coefficients_original": dict(zip(self.feature_names, original.astype(float))),
                "intercept_original": float(model.intercept_ - original @ scaler.mean_),
            }
        path = self.model_dir / f"cutoff-{cutoff:03d}.joblib"
        joblib.dump({"model": model, "scaler": scaler, "features": self.feature_names,
                     "model_kind": self.model_kind, "feature_mode": self.feature_mode}, path,
                     ↪  compress=3)
        record = {
            "strategy": self.name, "branch": self.f.branch, "model_kind": self.model_kind,
            "feature_mode": self.feature_mode, "window": self.window, "cutoff_day": int(cutoff),
            "cutoff_datetime": q4.daystr(cutoff) + " 00:00", "latest_target_end": q4.daystr(cutoff) +
            ↪  " 00:00",
            "training_rows": int(len(y)), "feature_names": self.feature_names,
            "model_path": str(path.relative_to(R)), "model_sha256": sha(path),
            "seconds": time.perf_counter() - started, **detail,
        }
        logs.event("price_fit_log", record)
        self.cache[cutoff] = model, scaler
        budget.save()
        return self.cache[cutoff]

    def predict(self, day, release):
        targets, net = self.f.supply_forecast(day, release)
        cutoff = 7 + ((day - 7) // 7) * 7
        model, scaler = self.fit(cutoff)
        full = self.f.matrix(day, release, targets, net)
        x, match = self.transform(full)
        pred = model.predict(x if scaler is None else scaler.transform(x))
        if self.model_kind != "p3":
            pred = match + pred
        pred = np.maximum(pred, 0.001)
```

```
        logs.event("price_forecast_log", {
            "strategy": self.name, "branch": self.f.branch, "decision_day": int(day),
            "decision_date": q4.daystr(day), "release_slot": int(release),
            "known_price_intervals_today": int(release), "fit_cutoff_day": int(cutoff),
            "forecast_points": int(len(pred)),
            "sha256": hashlib.sha256(np.asarray(pred, dtype=np.float64).tobytes()).hexdigest(),
        })
        return pred
```

q4_update.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False
 ↪  ,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
import q4
class Features(q4.PriceFeatures):
 def supply_forecast(self,day,slot):
  b=day*4+slot//36;n=288-slot
  return day*144+slot+np.arange(n),self.supply['load'][b,:n]-self.supply['pv'][b,:n]

class Prices(q4.PriceModels):
 def fit(self,candidate,cutoff_day):
  ans=super().fit(candidate,cutoff_day);self.datasets.clear();return ans
```

reproduce.py

```
"""Reproduce the paper's final schedules from prepared inputs; no parameter search."""
```

```
import os
for key in ('OMP_NUM_THREADS','OPENBLAS_NUM_THREADS','MKL_NUM_THREADS','NUMEXPR_NUM_THREADS'):os.environ[key]='1'
from pathlib import Path
import argparse,json
import numpy as np
import pandas as pd
from threadpoolctl import threadpool_limits
import q1,q2,q3_base,q3,q4,q4_tree,q4_update
ROOT=Path(__file__).resolve().parents[1]
SRC=dict(np.load(ROOT/'data/inputs.npz'))
OUT=Path(os.environ.get('SUPPORT_REPRO_OUTPUT',str(ROOT/'reproduced'))).resolve();OUT.mkdir(parents=True,exist_ok=True)

def save(label,order,execution,versions=None,qzero=None):
 arrays=dict(days=np.arange(31,31+len(order)),execution=execution)
 if versions is None:arrays['order']=order
 else:arrays.update(q0=qzero,versions=versions)
 np.savez_compressed(OUT/f'{label}.npz',**arrays)
 return arrays

def question1():
 cfg=json.loads((ROOT/'data/q1_config.json').read_text())
 ref=pd.read_csv(ROOT/'results/Q1.csv',float_precision='round_trip')
 # Only source inputs are reused; orders, states and costs are solved anew.
 inputs=ref[['k','start_minute','end_minute','load_kw','pv_kw','price']].to_dict('records')
 actions,certificate=q1.solve_lp(inputs,cfg);rows=q1.schedule(inputs,cfg,actions)
 pd.DataFrame(rows).to_csv(OUT/'Q1.csv',index=False,float_format='%.17g')
 assert abs(sum(r['cost_yuan'] for r in rows)-35126.85)<.005
 return np.array([r['purchase_kwh'] for r in rows])

def january(data,q1order):
 planner=q2.Planner(data.price,lambda *a:None,lambda:None);state=6000.;orders=[];ex=[]
 for d in range(31):
  order=q1order if d==0 else planner.solve(*data.history(d,1,3)[:2],state,np.zeros(288),{})[0][:144]
  l,v=data.truth(d,initialization=d==0);block,state=q3_base.execute_block(l,v,order,state)
  orders.append(order);ex.append(block)
 np.savez_compressed(OUT/'January.npz',days=np.arange(31),order=orders,execution=ex)
 assert abs(state-8592.693560925925)<1e-6,state
 return state

def question2(data,initial,end,quantile=.7,decay=.7):
 planner=q2.Planner(data.price,lambda *a:None,lambda:None);risk=q2.Risk(data);state=initial;orders=[];ex=[]
 for d in range(31,end):
  l,v,_=data.history(d,4,7);m,_=risk.margin(d,quantile,28,4,decay)
  order=planner.solve(l,v,state,m,{})[0][:144]
  a,b=data.truth(d);block,state=q3_base.execute_block(a,b,order,state);orders.append(order);ex.append(block)
 return save('Q2',np.array(orders),np.array(ex))

def supply3(family='EW4',quantile=.6,information='full'):
 actual,reference,F,release,targets=(SRC[k] for k in ['actual','reference','F','release','targets'])
 hist,last,L,H,X=q3_base.history_and_features(actual,reference,F,release,targets)
 hourly,w,count,fb,cut=q3_base.build_fusion(actual,F,H,release,targets)
 if information=='no_new':
  oldF=H.copy()
  for b,r in enumerate(release):
```

```
    midnight=r//144*144;t=targets[b];use=t<=midnight+144
    oldF[b,use]=F[(r//144)*4,((t[use]-midnight)//6-1).astype(int)]
   hourly,w,count,fb,cut=q3_base.build_fusion(actual,oldF,H,release,targets)
  full=q3_base.to_intervals(hourly,hist,last,release)
  q3.src=SRC;q3.orig={'L':L};q3.fc={'L':q3.midnight(family)}
  load,theta,obs,count=q3.make_load('LEARN_PREFIX',1.)
  err=q3.errors_for(load,full);q3.group_cache=None;margin=np.full((1460,288),np.nan)
  for b,typ,g,dest,ids,ages,fallback in q3.group_index(err):
   margin[b,dest]=q3.quantile(err['error'][ids],ages,quantile) if len(ids)>=24 else 0.
  np.savez_compressed(OUT/'supply3.npz',load=load,pv=full,margin=margin,weights=w,theta=theta)
  return load,full,margin

def question3(initial,end,supply,locked=False):
  load,pv,margin=supply;planner=q3.Planner(SRC['price'],q3.Budget());state=initial
  orders=[];zeros=[];versions=[];ex=[]
  for d in range(31,end):
   order=np.zeros(144);dayversions=[];blocks=[]
   for a,r in enumerate((0,36,72,108)):
    b=d*4+a;n=288-r
    if a==0 or not locked:
     plan,_=planner.solve(load[b,:n],pv[b,:n],state,margin[b,:n],r,None if a==0 else qzero[r:])
     order[r:]=np.maximum(0,plan[6,:144-r])
    if a==0:qzero=order.copy()
    dayversions.append(order.copy());truth=SRC['actual'][d*144+r:d*144+r+36]
    block,state=q3_base.execute_block(truth[:,0],truth[:,1],order[r:r+36],state);blocks.append(block)
   orders.append(order.copy());zeros.append(qzero);versions.append(dayversions);ex.append(np.concaten
    ↪ ate(blocks))
   if (d-30)%100==0:print('Q3 completed days',d-30,flush=True)
  return save('Q3',np.array(orders),np.array(ex),np.array(versions),np.array(zeros))

def question4(data,initial,end,supply,branch):
  for m in (q4,q4_tree,q4_update):m.R=OUT;m.O=OUT
  prices=SRC['actual_prices'];reference=SRC['price'];state=initial
  if branch=='Q4-2':
   load=np.empty((365,288));pv=load.copy();margin=load.copy();risk=q4.Q2Risk(data)
   load[0]=np.tile(SRC['reference'][:,0],2);pv[0]=np.tile(SRC['reference'][:,1],2);margin[0]=0
   for d in range(1,365):load[d],pv[d],_=data.history(d,4,7);margin[d]=risk.margin(d)[0]
   features=q4.PriceFeatures(prices,reference,dict(load=load,pv=pv),branch)
   engine=q4_tree.ForecastEngine('Q4-2-leaves31',features,'p3',max_leaf_nodes=31)
   planner=q4.Q42Planner(.9*reference[0]);orders=[];ex=[]
   for d in range(31,end):
    forecast=engine.predict(d,0);order=planner.solve(load[d],pv[d],state,margin[d],forecast,dict(day=
     ↪ d))[:144]
    a,b=data.truth(d);block,state=q4.execute_block(a,b,order,state);orders.append(order);ex.append(bl
     ↪ ock)
    if (d-30)%50==0:print(branch,'completed days',d-30,flush=True)
   return save(branch,np.array(orders),np.array(ex))
  load,pv,margin=supply
  features=q4_update.Features(prices,reference,dict(load=load,pv=pv),branch)
  engine=q4_update.Prices(features,branch);planner=q4.Q43Planner(.9*reference[0])
  orders=[];zeros=[];versions=[];ex=[]
  for d in range(31,end):
   order=np.zeros(144);dayversions=[];blocks=[]
   for a,r in enumerate((0,36,72,108)):
    b=d*4+a;n=288-r;forecast=engine.predict('P2',d,r)
    order[r:]=planner.solve(load[b,:n],pv[b,:n],state,margin[b,:n],r,None if a==0 else
     ↪ qzero[r:],forecast,dict(day=d,release_slot=r))[:144-r]
    if a==0:qzero=order.copy()
```

```
    dayversions.append(order.copy());truth=SRC['actual'][d*144+r:d*144+r+36]
    block,state=q4.execute_block(truth[:,0],truth[:,1],order[r:r+36],state);blocks.append(block)
   orders.append(order.copy());zeros.append(qzero);versions.append(dayversions);ex.append(np.concaten
   ↪  ate(blocks))
   if (d-30)%100==0:print(branch,'completed days',d-30,flush=True)
  return save(branch,np.array(orders),np.array(ex),np.array(versions),np.array(zeros))

def compare(label,z):
 old=np.load(ROOT/'results'/f'{label}.npz');n=len(z['days']);errors={}
 for k in ('order','q0','versions','execution'):
  if k in z:
   errors[k]=float(np.max(abs(z[k]-old[k][:n])))
   np.testing.assert_allclose(z[k],old[k][:n],rtol=0,atol=1e-5,err_msg=label+' '+k)
 print(label,'reproduced',n,'days; max differences',errors,flush=True)
 return errors

def main():
 p=argparse.ArgumentParser();p.add_argument('--question',choices=['all','Q1','Q2','Q3','Q4-2','Q4-3'
 ↪  ],default='all');p.add_argument('--days',type=int,default=334);a=p.parse_args()
 assert 1<=a.days<=334;end=31+a.days
 data=q2.Data(SRC);order=question1();initial=january(data,order);report={}
 if a.question in ('all','Q2'):report['Q2']=compare('Q2',question2(data,initial,end))
 supply=supply3() if a.question in ('all','Q3','Q4-3') else None
 if a.question in ('all','Q3'):report['Q3']=compare('Q3',question3(initial,end,supply))
 for branch in ('Q4-2','Q4-3'):
  if a.question in
  ↪  ('all',branch):report[branch]=compare(branch,question4(data,initial,end,supply,branch))
 (OUT/'reproduction_check.json').write_text(json.dumps(dict(days=a.days,errors=report),indent=2),enc
 ↪  oding='utf-8')
if __name__=='__main__':
 with threadpool_limits(limits=1):main()
```

validation.py

```
"""Reproduce the existing paper comparisons; choose one setting per invocation."""
import argparse,itertools,json
import numpy as np
import pandas as pd
import reproduce as r
from threadpoolctl import threadpool_limits
def main():
 p=argparse.ArgumentParser();p.add_argument('question',choices=['Q1','Q2','Q3']);p.add_argument('--q
 ↪  uantile',type=float);p.add_argument('--decay',type=float,default=.7);p.add_argument('--family',
 ↪  choices=['W1','W2','W4','EW4'],default='EW4');p.add_argument('--information',choices=['full','n
 ↪  o_new','zero_only'],default='full');p.add_argument('--days',type=int,default=334);a=p.parse_arg
 ↪  s()
 assert 1<=a.days<=334
 if a.question=='Q1':
  cfg=json.loads((r.ROOT/'data/q1_config.json').read_text());inputs=pd.read_csv(r.ROOT/'results/Q1.c
  ↪  sv',float_precision='round_trip')[['k','start_minute','end_minute','load_kw','pv_kw','price']]
  ↪  .to_dict('records');rows=[]
  for reserve in [False,True]:
   for low,high,target in itertools.product(cfg['low_quantiles'],cfg['high_quantiles'],cfg['target_f
   ↪  ractions_of_usable_capacity']):
    actions,_=r.q1.rule(inputs,cfg,low,high,target,reserve)
```

```
    rows.append(dict(reserve=reserve,low=low,high=high,target=target,**r.q1.metrics(r.q1.schedule(inputs,cfg,actions))))
  for initial in [1200,6000,10800]:
   cfg.update(initial_kwh=initial,terminal_kwh=initial);actions,_=r.q1.solve_lp(inputs,cfg);rows.append(dict(initial=initial,**r.q1.metrics(r.q1.schedule(inputs,cfg,actions))))
  pd.DataFrame(rows).to_csv(r.OUT/'Q1_comparisons.csv',index=False);print('Q1: 54 rule settings and three initial-storage settings reproduced.');return
 initial=r.january(r.q2.Data(r.SRC),r.question1())
 if a.question=='Q2':z=r.question2(r.q2.Data(r.SRC),initial,31+a.days,a.quantile if a.quantile is not None else .7,a.decay)
 else:
  supply=r.supply3(a.family,a.quantile if a.quantile is not None else .6,'no_new' if a.information=='no_new' else 'full')
  z=r.question3(initial,31+a.days,supply,a.information=='zero_only')
 ex=z['execution'];order=z['order'] if 'order' in z else z['versions'][:,-1];zero=z.get('q0',order);price=r.SRC['price']
 normal=float((order*price).sum());adjust=float((.5*price*abs(order-zero)).sum());emergency=float((5*price*ex[:,:,8]).sum())
 print(json.dumps(dict(settings=vars(a),normal_cost=normal,adjustment_cost=adjust,emergency_cost=emergency,total_cost=normal+adjust+emergency,emergency_kwh=float(ex[:,:,8].sum())),ensure_ascii=False,indent=2))
if __name__=='__main__':
 with threadpool_limits(limits=1):main()
```